Drawlib Quickstart Guide

Illustration as Code & Illustrated Documentation as Code in Pure Python

Drawlib unifies declarative pure-Python illustrations and Markdown technical writing into a single Git-native source of truth—compiling documentation portals (`site`), searchable API references (`catalog`), multi-chapter whitepapers (`doc`), interactive 16:9 presentations (`slide`), standalone image/animation suites (`images`), vector PDFs, and editable Microsoft Office deliverables (`.docx` & `.pptx`).

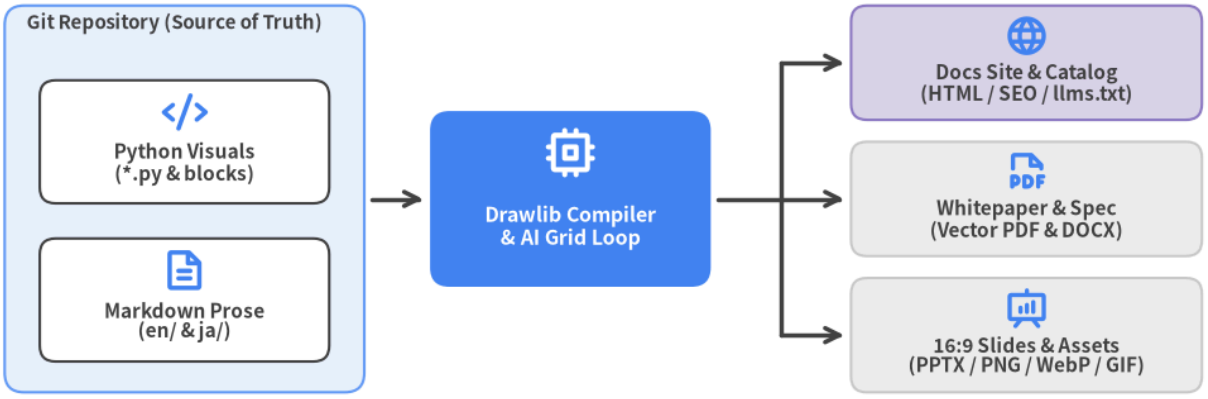


Figure 0.1: Drawlib Single Source of Truth — Declarative Python & Markdown to Multi-Target Deliverables

Document Overview & Roadmap

This 16-chapter handbook guides software engineers, technical writers, and AI coding agents from Drawlib fundamentals to multi-target publishing:

- **Chapters 1–3 (Foundations)**: Core motivation, zero-config installation (`uv` & `uvx`), and Cartesian `(0, 0)` canvas geometry.
- **Chapters 4–6 (Primitives, Styles & Icons)**: 43 geometric shapes, curved/orthogonal lines, `GoogleStyles` neutral token discipline, universal CJK/Latin typography, and 34 vector icon suites.

- **Chapters 7–11 (High-Level Visual Engines)**: 18 SmartArts, 10 quantitative charts (including `BoxChart` & `HeatmapChart`), `ArchitectureDiagram`, UML/software models, and 5 `drawlib.graph` automatic layout solvers.
- **Chapters 12–15 (Animations, Publishing & AI Loops)**: Multi-frame `APNG` / `WebP` / `GIF` animations, 16:9 slides, the 5 project archetypes (`site`, `catalog`, `doc`, `slide`, `images`), CLI caching, and autonomous 3-stage AI visual verification.

Document Metadata	Value
Drawlib Version	`0.3.10`
Visual Theme	`GoogleStyles` & `GoogleColors` (`style = "google"`)
Languages (`[i18n]`)	English (`en`) & Japanese (`ja`)
Official Website	`https://www.drawlib.com/`
Source Repository	`https://github.com/yuichi110/drawlib`
License	Apache License 2.0

Table of Contents

1. Why Drawlib?
2. Installation & Zero-Config Toolchain
3. Cartesian Canvas & Coordinate Anchors
4. Drawing Primitives: Shapes, Lines & Math
5. Styling System, Design Tokens & Universal Fonts
6. 34 Icon Suites & Media Assets
7. SmartArts: 18 Structured Visual Components
8. Quantitative Charts: 10 Pure-Python Chart Types
9. Cloud & System Architecture Diagrams
10. Software Design Diagrams: Sequence, Flow, State, Class & ER
11. Automatic Graph Layout Solvers
12. Multi-Frame Animations & 16:9 Slide Decks
13. Documentation as Code & The 5 Project Archetypes
14. CLI Toolchain & Autonomous AI Agent Workflows
15. Visual Design Rules & Production Checklist

1. Why Drawlib?

Modern software systems evolve at unprecedented speed through pull requests, automated CI/CD pipelines, and AI coding agents (Claude Code, Cursor, Gemini CLI, GitHub Copilot)—shifting the primary engineering bottleneck from writing implementation code to **visualizing system architecture and preventing "Documentation Rot"**. When diagrams and specs are maintained in disconnected manual tools, documentation quickly falls behind the codebase, slowing down onboarding and burying design intent inside ephemeral chat logs and commit histories.

Drawlib eliminates this disconnect by delivering **"Illustration as Code"** and **"Illustrated Documentation as Code" (IDaC)**: unifying declarative pure-Python visuals, inline Markdown embedding, and multi-format technical publishing inside a single Git repository so humans and AI coding agents can author, diff, verify, and publish living visual documentation as code.

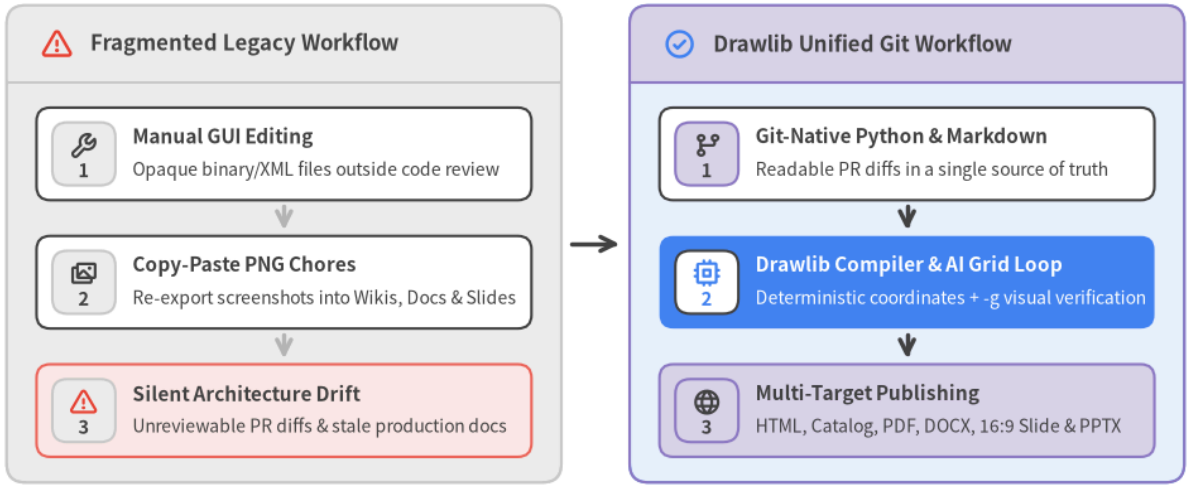


Figure 1.1: Fragmented Legacy Diagramming vs. Drawlib Unified Git Workflow

1.1 Why Existing Visual Tooling Fails Humans and AI Agents

Engineering teams typically piece together four workarounds for diagrams and technical visuals, each breaking down when maintaining production documentation with AI coding agents:

- **GUI Diagramming Tools & Binary PNGs (`draw.io`, `Figma`, `Visio`, `Slides`)**: Visual editors store diagrams as binary images (`PNG` / `JPEG`) or sprawling XML files. Binary images cannot be diffed in Git pull requests, cannot be inspected or edited by AI coding agents, vary wildly in font/color consistency across authors, and force manual re-exports into wikis, Word specs, and slide decks—guaranteeing silent documentation rot.
- **Text-to-Diagram DSLs (`Mermaid`, `PlantUML`, `Graphviz`)**: Domain-specific text syntaxes integrate cleanly into Markdown, but delegate spatial geometry to rigid black-box layout engines. Inserting a single dependency edge can unexpectedly reshuffle an entire topology, fine-grained brand typography and spacing controls are minimal, and native support for quantitative statistical charts (`BoxChart`, `HeatmapChart`) or rich SmartArts is absent.
- **Raw SVG Generation by AI Agents**: Asking an AI agent to emit raw `<svg>` markup directly fails in practice—computing multilingual (CJK/Latin) font metrics and text wrapping is error-prone (causing labels to overflow box borders), manual arrow routing produces crossed lines and misaligned arrowheads, and hundreds of low-level `<path>` and `<rect>` coordinates are impossible for humans to review or maintain.
- **Scientific Plotting Libraries (`Matplotlib`, `Seaborn`)**: Numeric plotting packages excel at statistical arrays, yet lack native high-level primitives for cloud architecture nodes, container groups, connector routing, standardized icon suites, UML/ER models, SmartArts, and multi-target document compilation. Assembling a clean microservice diagram requires hundreds of lines of low-level imperative boilerplate that frequently triggers AI coordinate and syntax errors.

Capability	GUI Suites (`draw.io` / `Visio`)	Text DSLs (`Mermaid` / `PlantUML`)	Raw SVG / `Matplotlib`	Drawlib (`v0.3.10`)
Git PR Diffability	Poor (opaque binary / noisy XML)	High (concise text)	Low (verbose `<path>` / boilerplate)	**High (declarative Python)**
Layout Control	Manual pixel dragging only	Black-box auto-layout only	Manual axis / pixel math	**Exact Cartesian + 5 Graph Solvers**

Capability	GUI Suites (`draw.io` / `Visio`)	Text DSLs (`Mermaid` / `PlantUML`)	Raw SVG / `Matplotlib`	Drawlib (`v0.3.10`)
Visual Scope	Diagrams & wireframes	Basic flow / sequence / ER	Low-level shapes or numeric plots	**Icons + SmartArts + Charts + Diagrams + Anim**
Publishing Targets	Manual image export	Embedded web blocks only	Standalone image files	**HTML, Catalog, PDF, `.docx` , Slide & `.pptx`**
AI Self-Correction	Unusable by coding agents	Blind to rendered overlaps	High CJK clipping & routing error rate	**Deterministic `-g` Grid Multimodal Loop**

1.2 The Three Pillars of Drawlib

Drawlib `v0.3.10` is engineered around three tightly integrated pillars that turn visual communication into a reproducible software engineering discipline.

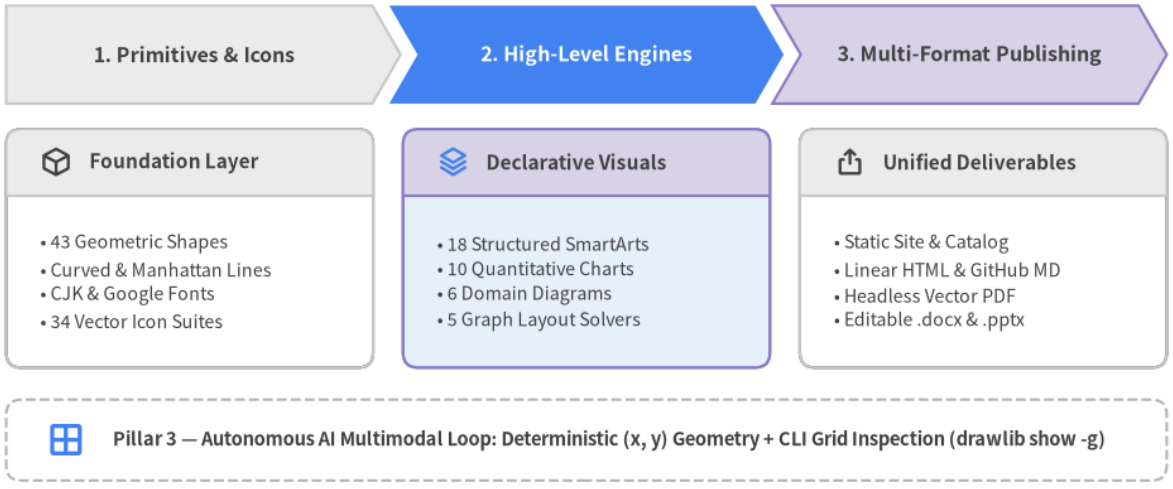


Figure 1.2: Drawlib Architecture — From Primitives & Visual Engines to Multi-Target Publishing

1.2.1 Pillar 1: Illustration as Code

Every visual asset in Drawlib is a pure-Python program backed by strict Pydantic type validation and reusable theme presets (`GoogleStyles` , `DefaultStyles` , `MonochromeStyles`):

- **Rich Composable Vocabulary**: Combine 43 geometric primitives, curved/orthogonal routed connectors, and **34 standardized vector icon suites** (`phosphor`, `gcp`, `aws`, `azure`, plus 15 domain and 15 flat original icon packs) with **18 SmartArts**, **10 quantitative charts**, **6 software/architecture diagram engines**, **5 automatic graph layout solvers** (`drawlib.graph`), and multi-frame **animations** (`APNG`, `WebP`, `GIF`).
- **Full Python Expressiveness**: Drive diagrams directly from real JSON schemas, OpenAPI specs, benchmark CSVs, or shared helper functions in `utils.py` rather than duplicating labels by hand.

1.2.2 Pillar 2: Illustrated Documentation as Code

Drawlib goes beyond standalone image generation by treating Markdown prose and embedded ```` ```drawlib ```` blocks as a single compilation unit:

- **Five Project Archetypes from One Toolchain**: Scaffold multi-page documentation portals (`site`), searchable API/CLI dictionaries (`catalog`), linear engineering whitepapers (`doc`), interactive `16:9` widescreen decks (`slide`), or standalone asset suites (`images`).
- **Multi-Format & Multi-Locale Publishing**: Compile localized `en/` and `ja/` sources with structural skeleton parity checks into responsive HTML (with SEO metadata and `llms.txt`), GitHub-Flavored Markdown, headless Chromium vector PDFs, editable Microsoft Word documents (`.docx`), and native PowerPoint decks (`.pptx`)—cached incrementally in `.drawlib/cache.db`.

1.2.3 Pillar 3: Autonomous AI Multimodal Loop

Drawlib is purpose-built from the ground up for collaboration between software engineers and multimodal AI coding agents:

- **Deterministic Coordinates + `-g` Grid Inspection**: Unlike black-box layout DSLs where agents cannot adjust a specific overlapping label, Drawlib's Cartesian coordinate plane and `drawlib show ... -g` (`10`-unit / `5`-unit grid overlay) give AI agents exact spatial coordinates.
- **Closed-Loop Visual Self-Correction**: An AI agent can inspect your codebase, author declarative Drawlib blocks, export a `-g` grid preview, inspect the rendered PNG

multimodally to spot any text collision or margin imbalance, and self-correct coordinates in seconds—assisted by built-in documentation via `drawlib rules show`.

2. Installation & Zero-Config Toolchain

Drawlib `v0.3.10` is packaged as a self-contained pure-Python toolchain for **Python** `3.11+` (`3.11`, `3.12`, `3.13`, **and** `3.14`) across Linux, macOS, and Windows—requiring zero external Graphviz binaries, LaTeX distributions, or Node.js bundlers.

Whether you are adding visual documentation to an existing Python package with `uv add drawlib` or authoring architecture specs inside a Go, Rust, TypeScript, or Java repository via zero-install `uvx drawlib`, the identical CLI and asset cache work out of the box.

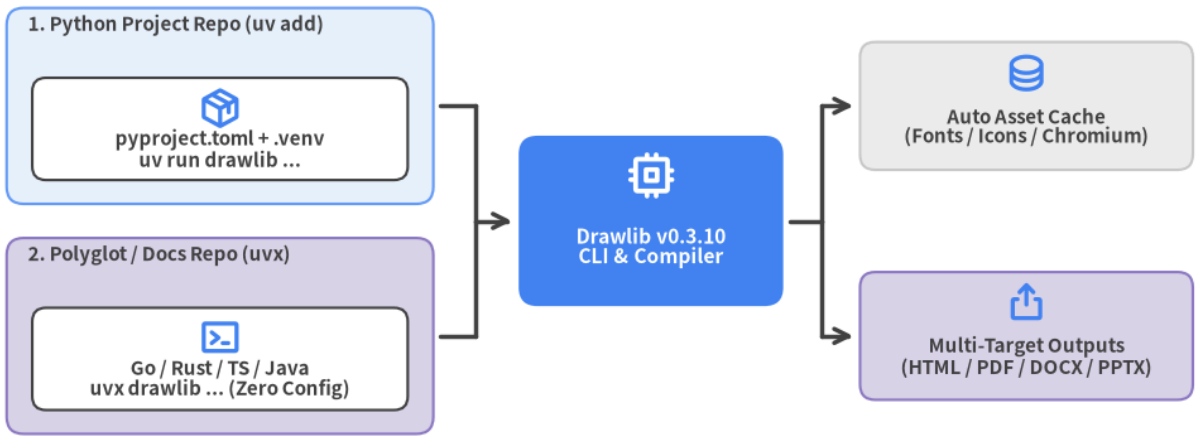


Figure 2.1: Two Execution Workflows — Project Dependency (uv add) vs. Zero-Install Polyglot Runner (uvx)

2.1 System Requirements & Execution Modes

Drawlib targets modern CPython releases and pairs naturally with Astral's `uv` package manager for deterministic lockfiles and instant CLI execution:

Requirement	Supported Specification	Notes
Python Runtime	**Python** `3.11+` (`3.11`, `3.12`, `3.13`, `3.14`)	Verified across Linux (`x86_64` / `aarch64`), macOS, and Windows

Requirement	Supported Specification	Notes
Recommended Runner	`uv` (`uv add` / `uv run` / `uvx`)	Instant virtual environments and zero-install polyglot execution
Standard Fallback	`pip` (`venv` + `pip install drawlib`)	Standard PyPI wheel installation without optional extra flags

2.1.1 Workflow 1: Python Project Dependency (`uv add drawlib`)

For Python services, data science repositories, or dedicated documentation repos where you want `drawlib` pinned inside `pyproject.toml` and `uv.lock`:

```
# Initialize a new uv project (or run inside an existing Python repository)
uv init my_project
uv add drawlib
uv run drawlib --version

# Standard pip + venv fallback when uv is unavailable
python3 -m venv .venv
.venv/bin/pip install drawlib
.venv/bin/drawlib --version
```

Pinning `drawlib` in `pyproject.toml` gives IDEs (`Pyright`, `Pylance`, `Ruff`) full autocomplete and static type checking across `drawlib.shapes`, `drawlib.smartarts`, `drawlib.charts`, and `drawlib.diagrams`, while guaranteeing reproducible CI builds via `uv sync --frozen`.

2.1.2 Workflow 2: Zero-Install Polyglot Runner (`uvx drawlib`)

For **Go, Rust, TypeScript, Java, C++, or pure Markdown repositories**, you do not need to pollute the repository root with a `pyproject.toml` or `.venv/` folder. Prefix any CLI command with `uvx` to run Drawlib inside an isolated, automatically cached tool environment:

```
# Verify Drawlib immediately with zero local Python configuration
uvx drawlib --version

# Scaffold and build multi-locale documentation inside any Go / Rust / TS repo
uvx drawlib init project site docs -l en,ja -s google
uvx drawlib build html docs_src/ -o docs_html/
```

2.2 Built-In Batteries & Asset Caching

Unlike fragmented documentation stacks that require separate plugins for diagrams, Word exports, slide decks, and PDF printing, `drawlib v0.3.10` **ships with every rendering and publishing dependency built in**—no `drawlib[pdf]` or `drawlib[office]` extras required.

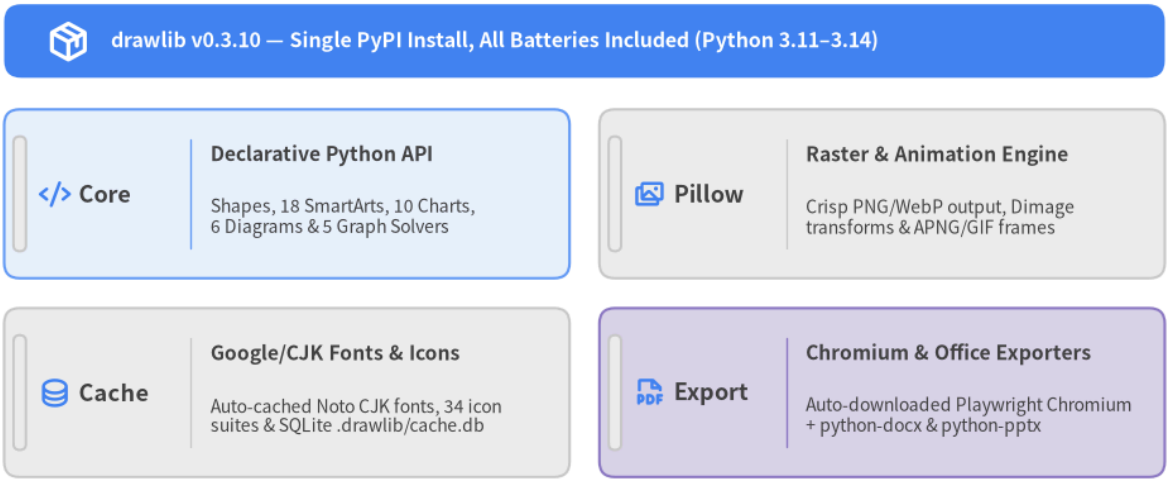


Figure 2.2: Drawlib v0.3.10 Zero-Config Runtime Stack & Automated Cache Architecture

Every `pip install drawlib` or `uv add drawlib` installation provisions four integrated subsystems:

- **Pillow Raster & Animation Engine (`pillow`)**: Powers high-DPI `PNG` and `WebP` rasterization, `Dimage` cropping/compositing transforms, and multi-frame animation encoding (`APNG`, animated `WebP`, and `GIF`) alongside Drawlib's vector geometry renderer.
- **Playwright Headless Chromium (`playwright`)**: Bundled as a standard dependency for pixel-accurate vector `PDF` (`drawlib build pdf`) and `16:9` slide rendering (`drawlib build pptx`). On your first PDF or PPTX build, Drawlib automatically downloads `chromium-headless-shell` with zero manual setup.
- **Native Microsoft Office Builders (`python-docx` & `python-pptx`)**: Compiles linear `doc` projects into editable `.docx` specifications and `16:9` `slide` decks into editable `.pptx` presentations out of the box.

- **On-Demand Font, Icon & SQLite Build Caches**: Standardized Google Fonts, **Noto Sans CJK** multilingual fonts (eliminating tofu boxes across Linux/macOS/Windows), and all 34 icon suites download automatically on first use, while `.drawlib/cache.db` hashes rendered blocks so unchanged diagrams skip re-execution.

For offline environments, air-gapped runners, or minimal Docker CI images (such as `python:3.12-slim`), pre-warm the asset cache ahead of time:

```
# Inspect local font, icon, map, and browser cache status
uv run drawlib cache list

# Pre-download all fonts (including CJK), 34 icon suites, maps & headless Chromium
uv run drawlib cache download --all

# Pre-download only headless Chromium for PDF / PPTX builds
uv run drawlib cache download --browser

# On minimal Linux CI containers missing OS shared libs (libnss3, libgbm1, etc.)
uv run playwright install-deps chromium
```

2.3 Quick-Test Project Scaffolding

Never hand-craft bare `.py` files or ad-hoc folders from scratch—always start with `drawlib init project` so `drawlib.toml`, `styles.py`, `utils.py`, `_assets/`, and modular `build*.sh` scripts are configured cleanly:

```
# 1. Confirm CLI version
uv run drawlib --version

# 2. Scaffold a bilingual (en + ja) linear document project with the Google theme
uv run drawlib init project doc my_docs -l en,ja -s google

# 3. Compile HTML, vector PDFs, GitHub Markdown, and extracted images
./my_docs_src/build.sh
```

Running `drawlib init project doc my_docs -l en,ja -s google` creates `my_docs_src/` containing `drawlib.toml` (`[project]` + `[i18n]`), `styles.py` (`GoogleStyles`), `ja/styles.py` (`FontJapanese` override), `utils.py`, `_assets/`, starter chapters under `en/` and `ja/`, and ready-to-run build scripts (`build.sh`, `build_html.sh`, `build_pdf.sh`, `build_docx.sh`, and `serve.sh`).

3. Cartesian Canvas & Coordinate Anchors

Every Drawlib illustration is composed on a deterministic **2D Cartesian coordinate plane** where the origin `(0, 0)` sits strictly at the **bottom-left corner**, `X` increases horizontally to the right (`0 -> width`), and `Y` increases vertically upward (`0 -> height`).

Mastering three foundational rules—**bottom-left Cartesian axes**, the `720pt` **typography formula**, and **component coordinate anchors**—lets engineers and AI coding agents place shapes, icons, charts, and SmartArts with single-shot precision and zero visual collisions.

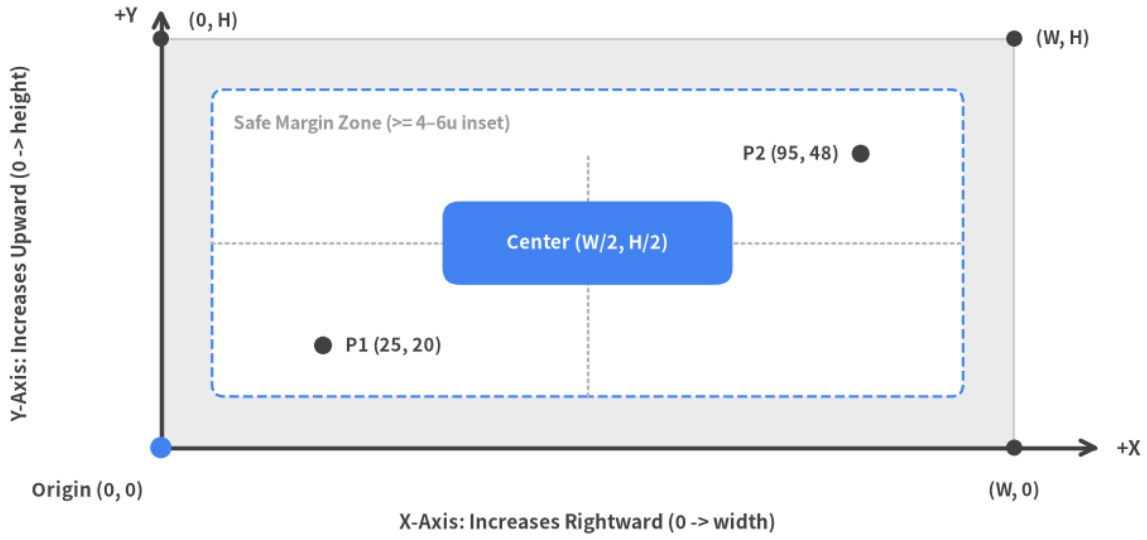


Figure 3.1: Drawlib Bottom-Left (0, 0) Cartesian Canvas, Safe Margin Zone, and Plotted Coordinates

3.1 Cartesian Coordinates vs. Browser DOM / SVG

Frontend engineers accustomed to HTML5 `<canvas>`, CSS absolute positioning, or raw `<svg>` must note one crucial geometric distinction so `Y` coordinates are never inverted:

Coordinate Property	Drawlib Cartesian Canvas	Browser DOM / HTML `<svg>`
Origin `(0, 0)`	**Bottom-Left corner**	Top-Left corner
Horizontal `+X` **Direction**	Left to Right (`0 -> width`)	Left to Right (`0 -> width`)
Vertical `+Y` **Direction**	**Bottom to Top (`0 -> height`)**	Top to Bottom (`0 -> height`)
Top Title Placement	High `Y` near `height - 6`	Small `Y` near `16px`
Safe Outer Margin	Keep content inside `[5, W - 5]` × `[5, H - 5]`	CSS padding / viewBox

Because `+Y` points upward just like mathematical plots and architectural elevation drawings, **higher `Y` always means visually higher on the page**—page titles sit near `height - 6`, central architecture tiers sit around `height / 2`, and footnotes or legends sit near `y = 6..10`. Always maintain `4–6` **virtual units of clear outer margin** around all four canvas edges so borders, rounded corners, and drop shadows never clip.

3.2 Canvas Lifecycle: `setup()`, Drawing Calls, and `save()`

Every Drawlib script and embedded Markdown block follows a clean three-step execution lifecycle:

1. `setup(width=..., height=..., dpi=..., background_color=..., grid=...)`: Initializes canvas geometry in **abstract virtual units** (`width` and `height`, defaulting to `100 × 100`) and configures raster output density via `dpi` (default `100`). Because coordinates use resolution-independent virtual units, increasing `dpi=100` to `dpi=200` doubles output pixel dimensions (`1400 × 520 px` -> `2800 × 1040 px`) without changing a single `(x, y)` number. Project-wide theme palettes and fonts (`from drawlib.styles import Colors, Styles`) load automatically from `styles.py` (`style = "google"` in `drawlib.toml`).
2. **Declarative Drawing Calls**: Invoke Z-ordered primitives, icons, SmartArts, charts, or domain diagrams from back (containers and connector lines) to front (service cards and callouts).
3. `save()`: Commits and renders the canvas. Always end **every standalone `.py` script and every embedded ```` ```drawlib ```` Markdown block** with `save()`—in standalone scripts `save()` flushes the image file to disk (`<script_name>.png` or an explicit path), and in Markdown blocks ending with `save()` ensures every snippet is 100% self-contained and

directly runnable outside the document builder. *(If a single `.py` script generates multiple sequential files, call `clear()` after each `save(...)` before the next `setup(...)`.)*

```
from drawlib.canvas import save, setup
from drawlib.lines import line
from drawlib.shapes import rectangle
from drawlib.styles import Styles

# 1. Abstract 136x50 virtual units at crisp 150 DPI (2040x750 output pixels)
setup(width=136, height=50, dpi=150)

# 2. Back-to-front drawing calls (neutral baseline + 1 focal PrimaryFlat node)
line((44, 25), (54, 25), arrow_head="->", style=Styles.DarkBold)
line((82, 25), (92, 25), arrow_head="->", style=Styles.DarkBold)

rectangle((26, 25), width=36, height=18, style=Styles.Neutral, text="Client App")
rectangle(
    (68, 25),
    width=28,
    height=18,
    style=Styles.PrimaryFlat,
    text="API Core",
    text_style=Styles.WhiteBold,
)
rectangle((110, 25), width=36, height=18, style=Styles.SecondaryNeutral,
text="PostgreSQL")

# 3. Mandatory lifecycle commit
save()
```

3.3 The `720pt` Typography Formula & Canvas Sizing

Internally, Drawlib normalizes figure typography against a fixed `720pt` **reference width**. Regardless of `dpi`, the visual font size of any label when an image is displayed at `Display Width` pixels in HTML, PDF, or slides is governed by one exact formula:

```
Rendered Font Size (px) = text_size × (Display Width (px) / 720)
```

Understanding this linear scaling relationship prevents microscopic, unreadable diagram labels:

- **Keep `text_size >= 10.0` Everywhere**: In HTML and A4 PDF documentation (`doc`, `site`, `catalog`), content columns typically span `~820px–900px` (`Display Width / 720 ≈`

`1.15–1.25`). Setting `text_size = 10.5–12.0` for body/node labels and `12.0–14.0` for headers produces crisp `~12px–17px` lettering that matches surrounding `16px` HTML prose. Never drop below `text_size = 10.0` (with `9.5` strictly reserved as an absolute floor for minor chart ticks).

- **Ideal Embedded Figure Width (`width=120..150`)**: Calling `setup(width=140, height=60)` (or `width=120..150`) gives clean `10`-unit grid arithmetic for multi-column diagrams while keeping aspect ratios compact above the fold.
- **Never Cap Markdown Fences with Narrow Pixel Widths**: Never add `width=500px` or `width=600px` to ```` ```drawlib ```` fences—shrinking `Display Width` to `500px` reduces `Display Width / 720` to `0.69`, shrinking every label inside the figure by over `30%`.

Target Context	Typical `setup(width, height)`	Display Width	Recommended `text_size`	Resulting Visual Size
Doc / Site Hero & Flow	`140 × 60` (`120..150` wide)	`~840px`	`10.5–12.0` (headers `12.0–14.0`)	`~12.5px–16.5px` (matches `16px` prose)
Dense Multi-Tier Schema	`148 × 72`	`~840px`	`10.0–11.5` (headers `12.0–13.5`)	`~11.7px–15.8px`
16:9 Full-Width Slide	`160 × 80`	`~1760px`	`11.0–13.0` (headers `14.0–16.0`)	`~27px–39px` (matches `26px` slide text)

3.4 Text Alignment & Component Coordinate Anchors

When placing standalone labels with `text(xy, label, style=...)`, the anchor point `xy` is controlled via `.patch(halign=..., valign=...)` on preset `Styles.*` tokens (never construct raw `Style(...)` instances directly):

- `halign="left" | "center" | "right"`: Controls whether text extends rightward from `x` (`"left"`), centers around `x` (`"center"`, default), or ends flush against `x` (`"right"`).
- `valign="bottom" | "center" | "top"`: Controls whether text sits above `y` (`"bottom"`), centers vertically on `y` (`"center"`, default), or hangs downward from `y` (`"top"`).

Beyond `text()`, Drawlib components follow **three predictable coordinate anchor conventions** summarized below:

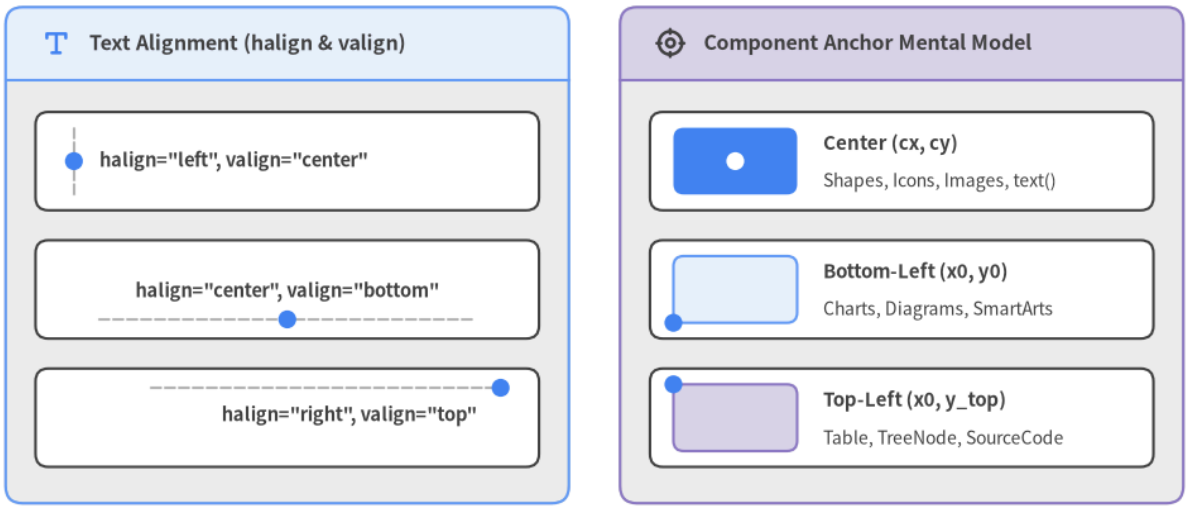


Figure 3.2: Text Alignment (halign / valign) and the Three Component Anchor Conventions

---

Keep this anchor mental model handy whenever combining primitives with composite SmartArts or Charts:

- **Center** `(cx, cy)`: Used by all geometric primitives (`shapes.*`), `text()`, icons (`phosphor.*`, `gcp.*`, `aws.*`, `azure.*`), `images.image()`, `MindMapNode`, `Cycle`, and individual `Node` items added inside `ArchitectureDiagram`. The element spans `[cx - w/2, cx + w/2]` horizontally and `[cy - h/2, cy + h/2]` vertically.
- **Bottom-Left** `(x0, y0)`: Used by all `charts.*`, `diagrams.*` (`d.draw(xy=(x0, y0))` and `NodeGroup(..., xy=(x0, y0))`), and most `smartarts.*` (`Panel`, `CardList`, `ChevronProcess`, `GridLayout`, `KpiCards`, `BoxList`, `Pyramid`, `Roadmap`, `GeoMap`, `*Calendar`). To center a component of width `w` on a canvas of width `W`, pass `x0 = (W - w) / 2` (never `W / 2`).
- **Top-Left** `(x0, y_top)`: Used by top-down document SmartArts—`Table`, `TreeNode`, `BulletPoints`, and `SourceCode`—so multi-row tables, directory trees, and code listings naturally expand downward from a fixed header line `y_top`.

4. Drawing Primitives: Shapes, Lines & Math

While high-level engines (`ArchitectureDiagram` , `SmartArts` , and `Charts`) automate full multi-node layouts, `drawlib.shapes` , `drawlib.lines` , and `drawlib.math` give you surgical vector control over custom badges, callouts, container geometry, and interpolated connector routes.

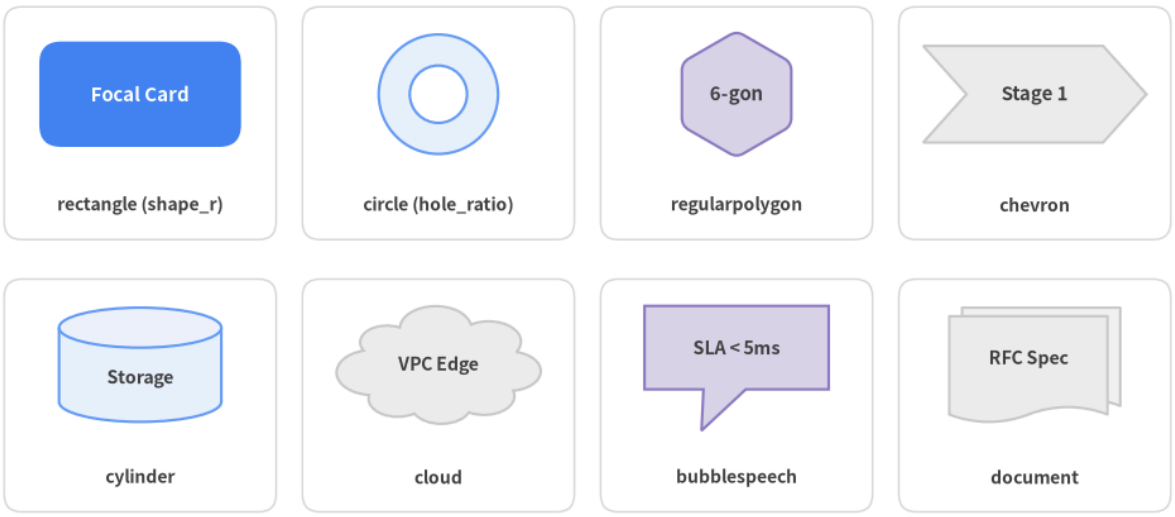


Figure 4.1: Representative Primitives Across Drawlib's 43 Geometric Shapes

4.1 The 43 Shapes of `drawlib.shapes`

`drawlib.shapes` organizes **43 pure-Python vector primitives** into 6 domain families. Every shape function enforces a keyword-only `*, style: Style` contract so geometry and visual tokens never mix.

Family	Count	Functions in `drawlib.shapes`	Typical Engineering Use Cases
Basic Shapes	6	`rectangle`, `circle`, `ellipse`, `arc`, `fan`, `wedge`	Service cards (`rectangle` + `shape_r`), nodes, donut/ring gauges (`circle` with `hole_ratio` or `wedge` with `width`), radial sectors
Polygons & Custom Paths	9	`triangle`, `rhombus`, `parallelogram`, `trapezoid`, `regularpolygon`, `star`, `plaque`, `polygon`, `shape`	Flowchart decisions (`rhombus`) & I/O (`parallelogram`), pentagons/hexagons/octagons (`regularpolygon(xy, n, r)`), arbitrary cutouts (`polygon` + `holes`)
Arrows & Chevrons	6	`arrow`, `arrow_polyline`, `arrow_arc`, `arrow_l`, `arrow_u`, `chevron`	Prominent phase banners (`chevron`), 1-way or 2-way block arrows (`arrow(..., head="<->")`), orthogonal L/U turn blocks
Containers & Cylinders	6	`cylinder`, `cube`, `document`, `funnel`, `cloud`, `brace`	Persistent databases (`cylinder` with `disks=2`), 3D nodes (`cube`), wave/folded specs (`document`), cloud boundaries, curly `brace` groupings
Badges & Callouts	8	`bubblespeech`, `ribbon`, `shield`, `tag`, `pin`, `checkmark`, `forbidden`, `explosion`	Inline speech callouts (`bubblespeech`), security/IAM badges (`shield`), release tags (`tag`, `ribbon`), geo markers (`pin`), pass/block indicators
Symbols & Flowchart Operators	8	`clock`, `face`, `gear`, `heart`, `lightning`, `math_symbol`, `moon`, `sun`	Cron/latency timers (`clock`), worker daemons (`gear`), cross/plus/arithmetic operators (`math_symbol`), event triggers (`lightning`)

When building standard software flowcharts, standard ISO symbols map directly to these primitives (`rhombus` for branching decisions, `parallelogram` for data I/O, `document` for artifacts, `cylinder` for storage, and `rectangle` with `shape_r` for process steps)—or can be automated end-to-end with `drawlib.diagrams.flow.FlowDiagram`.

4.2 Center Anchors `(cx, cy)`, Inline Text, Corner Radius & Rotation

Unlike top-left DOM boxes or bottom-left chart frames, **planar shapes in `drawlib.shapes` anchor at their exact geometric center** `xy = (cx, cy)` (with explicit-vertex exceptions `arrow(xy1, xy2, ...)`, `arrow_polyline(xys, ...)`, `polygon(xys, ...)`, and `bubblespeech(xy, ...)` whose body box starts at bottom-left `(x0, y0)`). This center-first convention makes aligning circles, hexagons, cylinders, and cards along a shared row `y = 20` effortless without subtracting half-widths.

Three ergonomic features keep primitive code compact and declarative:

- **Built-in Inline Labels (`text` & `text_style`)**: 39 of the 43 shapes render centered text directly via `text="..."` and `text_style=...` (eliminating extra `text()` calls). Use `text_style=Styles.DarkBold.patch(xy_abs_shift=(0, -12))` to place captions cleanly underneath compact badges like `pin` or `gear`.
- **Smooth Corner Rounding (`shape_r`)**: Patch `shape_r=2.5` onto `style` to round polygon corners with quadratic Bézier fillets on `rectangle`, `triangle`, `rhombus`, `parallelogram`, `trapezoid`, `regularpolygon`, `star`, and `polygon`.
- **Counter-Clockwise Rotation (`angle`) & Hollow Cutouts (`hole_ratio`)**: Patch `angle=30` onto `style` to rotate a shape CCW around `(cx, cy)` (pass `angle=0` on `text_style` to keep its label horizontal), or pass `hole_ratio=0.5` to cut a true transparent compound ring.

```
from drawlib.canvas import save, setup
from drawlib.shapes import circle, rectangle, regularpolygon
from drawlib.styles import Styles

setup(width=120, height=40)

# Center-anchored at y=20 with inline text, corner radius, and rotation
rectangle(
    (25, 20),
    width=30,
    height=16,
    style=Styles.PrimaryFlat.patch(shape_r=2.5),
    text="Ingress",
    text_style=Styles.WhiteBold.patch(text_size=11.0),
)
regularpolygon(
    (65, 20),
    6,
    10.0,
    style=Styles.PrimaryNeutral.patch(angle=15, shape_r=1.0),
    text="Router",
    text_style=Styles.DarkBold.patch(text_size=10.5, angle=0),
)
circle(
    (100, 20),
    radius=9.5,
    hole_ratio=0.5,
    style=Styles.Neutral,
    text="50%",
    text_style=Styles.DarkBold.patch(text_size=10.5),
)

save()
```

4.3 Vector Connectors (`drawlib.lines`) & Geometry Helpers (`drawlib.math`)

To wire shapes together cleanly, `drawlib.lines` provides 1D stroke connectors while `drawlib.math` supplies pure-Python vector and interpolation math so you never have to hardcode fragile trigonometry.

- **Straight & Polylines (`line`, `lines`, `lines_curved`):** `line(xy1, xy2, *, style, arrow_head="->")` draws a direct segment, `lines(xys, ...)` draws sharp orthogonal Manhattan routes, and `lines_curved(xys, r=3.5, ...)` rounds every interior polyline corner with fillet radius `r` (keep $r < \frac{1}{2}\min |P_{i+1} - P_i|$). Every line function accepts `arrow_head` values `""`, `"-"`, `"->"`, `"<-"`, and `"<->"`.
- **Arcs & Bézier Curves (`line_curved`, `line_bezier1`, `line_bezier2`, `lines_bezier`, `line_arc`):** `line_curved(xy1, xy2, bend=0.22, ...)` bulges smoothly left (`bend > 0`) or right (`bend < 0`) of the travel vector—ideal for request/response pairs—while `lines_bezier(xy, path_points, ...)` chains straight `(x, y)`, quadratic `(cp, end)`, and cubic `(cp1, cp2, end)` segments in a single path.
- **Coordinate & Trajectory Math (`drawlib.math`):**
  - `get_intermediate_point(xy1, xy2)` & `get_intermediate_points(xy1, xy2, num=3)`: Sample evenly spaced waypoints along a segment (plus `get_intermediate_path_points(xys, num=...)` along multi-corner polylines).
  - `rotate_point(xy, angle, center)` & `get_point_on_ellipse(center, width, height, angle)`: Compute exact radial node positions and boundary anchor points without manual `sin` / `cos` boilerplate.
  - `get_angle(xy1, xy2)`, `get_distance(xy1, xy2)`, `get_center_and_size(xys)`, & `compute_orthogonal_path(...)`: Measure slopes, auto-fit cluster bounding boxes, and synthesize L/Z-shaped orthogonal routes.

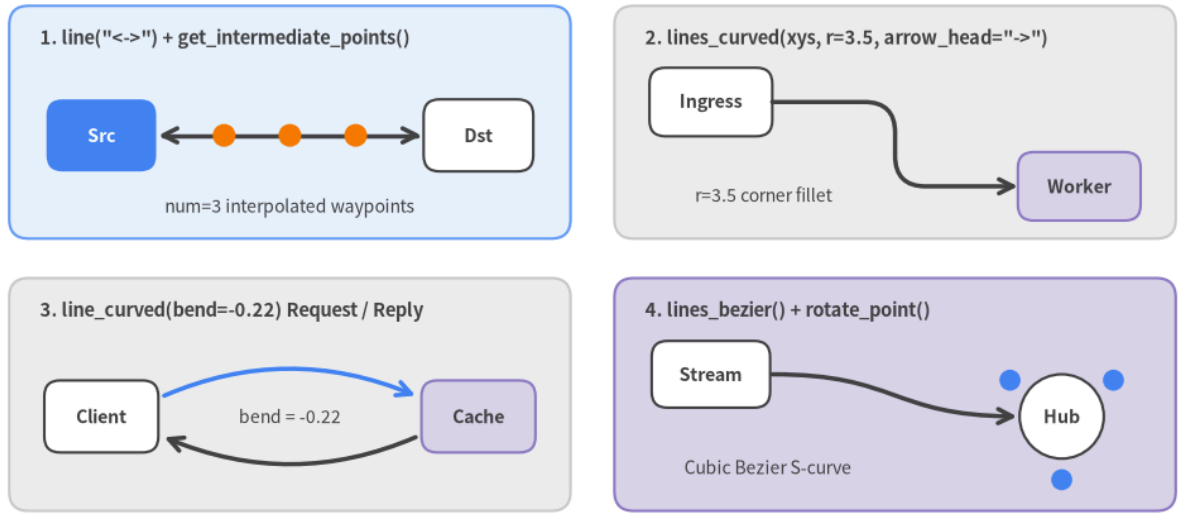


Figure 4.2: Vector Connectors (line, lines_curved, line_curved, lines_bezier) and Math Waypoint Helpers

5. Styling System, Design Tokens & Universal Fonts

Drawlib decouples coordinate geometry from visual presentation through immutable `Style` and `Color` models, semantic design tokens (`Styles.*` and `Colors.*`), and an on-demand multilingual font engine (`drawlib.fonts`) that renders Latin, monospace, and CJK scripts identically across every machine.

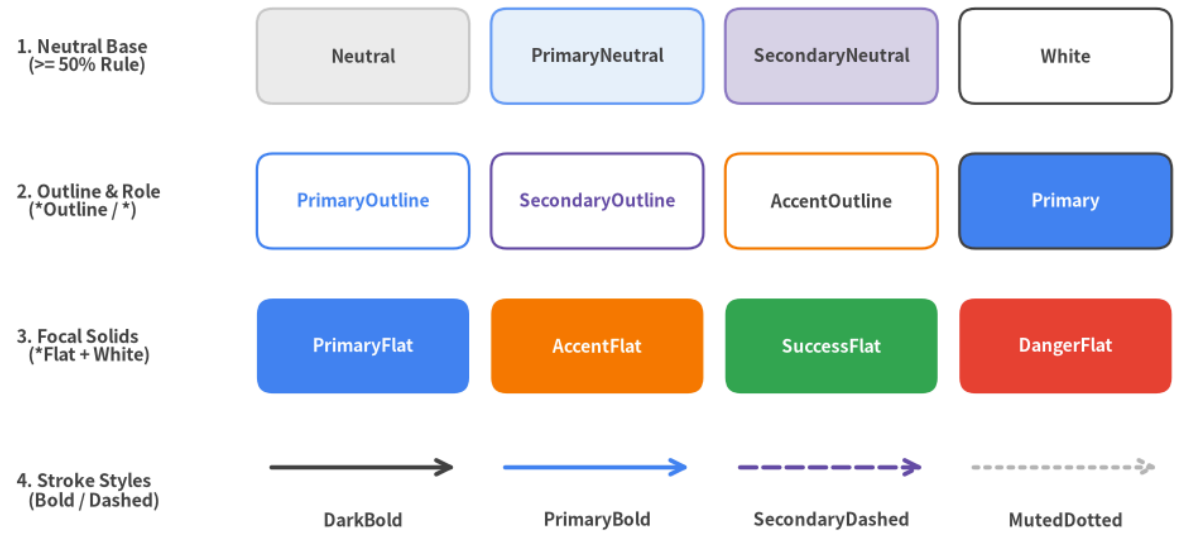


Figure 5.1: GoogleStyles Token Matrix — Neutral Surfaces, Outline/Role Tokens, Focal Solids, and Stroke Variants

5.1 Dynamic Theme Injection: `drawlib.styles` vs. `preset_styles`

In every script and Markdown `drawlib` block, always import **PascalCase** `Colors` **and** `Styles` from `drawlib.styles` :

```python
from drawlib.styles import Colors, Styles
```

Never import lowercase `styles` or `colors` (which shadows the `drawlib.styles` module), and avoid hardcoding concrete catalogs like `GoogleStyles` inside diagram files. Instead, `drawlib.styles` acts as a dynamic runtime facade populated by your project's root `styles.py`—and optionally overridden per language by `<lang>/styles.py` (such as `ja/styles.py` for Japanese typography):

- `DefaultStyles` / `DefaultColors`: Balanced 6-tone chromatic palette (`DefaultStyles1` through `DefaultStyles6`) tailored for technical documentation.
- `GoogleStyles` / `GoogleColors`: Material and Google Workspace presentation palette (`style = "google"` in `drawlib.toml`, used throughout this guide).
- `MonochromeStyles` / `MonochromeColors`: High-contrast 8-step grayscale theme for print publications, academic papers, and patent drawings (plus `CssColors` for all 140 W3C named web colors).

Because every theme exposes the exact same semantic token names, switching `styles.py` from `GoogleStyles()` to `MonochromeStyles()` restyles hundreds of diagrams without changing a single line of diagram code.

5.2 Semantic Token Families & The `50%+` Neutral Discipline

Professional architecture diagrams rely on visual hierarchy rather than rainbow fills. Every built-in theme organizes tokens into semantic roles and orthogonal variant suffixes:

Token Family / Suffix	Examples	Visual Behavior & Contrast Rule
Neutral Foundations	`Neutral`, `PrimaryNeutral`, `SecondaryNeutral`, `White`	Calm light/tinted card fills with subtle borders. **Ground `>= 50%` of shapes and containers in these tokens.** Always pass `text_style=Styles.DarkBold` when labeling `Styles.White`.
Semantic Roles	`Primary`, `Secondary`, `Accent`, `Warning`, `Success`, `Danger`, `Muted`	Blue (`Primary`), Purple/Indigo (`Secondary`), Orange (`Accent`), Green (`Success`), Red (`Danger`), and Gray (`Muted`), plus numbered tone scales (`Primary1` light tint through `Primary6` deep shade).

Token Family / Suffix	Examples	Visual Behavior & Contrast Rule
Outline / Wireframe (`*Outline` / `*Solid`)	`PrimaryOutline` , `SecondaryOutline` , `AccentOutline`	Transparent interior with a solid colored border (`1.5` width), ideal for logical zones, VPC frames, and lightweight badges.
Focal Solids (`*Flat`)	`PrimaryFlat` , `AccentFlat` , `SuccessFlat` , `DangerFlat`	Borderless saturated fills (`shape_line_width=0`). **Reserve strictly for 1–2 focal elements per diagram and pair with** `Styles.WhiteBold` .
Weight & Pattern Suffixes	`*Bold` , `*Thin` , `*Dashed` , `*Dotted`	Adjust stroke width (`Bold=2.5` , `Thin=0.75` , plus standalone `Styles.Light`) and dash pattern (`MutedDashed` , `SecondaryDashedBold` , `PrimaryDotted`).

5.3 Immutable `.patch()` , `.patch_font()` & Universal Fonts

Every `Style` and `Color` instance is **strictly immutable**. To customize corner rounding, stroke thickness, alignment, font size, or opacity, call `.patch(...)` on an existing `Styles.*` preset rather than constructing a bare `Style(...)` from scratch. Deriving from `Styles.*` preserves coordinated font families, icon weights, arrow fills, and theme-switching compatibility:

```
from drawlib.canvas import save, setup
from drawlib.fonts import FontFile, FontJapanese, FontMonoSpace
from drawlib.preset_styles import GoogleStyles
from drawlib.shapes import rectangle
from drawlib.styles import Colors, Styles

# Locale-level font override pattern used in ja/styles.py:
JaStyles = GoogleStyles().patch_font(
    regular=FontJapanese.SANSSERIF_REGULAR,
    bold=FontJapanese.SANSSERIF_BOLD,
    thin=FontJapanese.SANSSERIF_THIN,
)
# Custom local .ttf/.otf files also work via FontFile("_assets/brand.ttf")
_ = FontFile

setup(width=120, height=36)

card_style = Styles.PrimaryNeutral.patch(
    shape_r=2.8,
    shape_line_width=2.0,
    alpha=0.95,
)
mono_label = Styles.DarkBold.patch(
    text_font=FontMonoSpace.ROBOTO_MONO_BOLD,
    text_size=11.0,
    halign="center",
    valign="center",
)
glass_tint = Colors.Primary.patch(alpha=0.15)
_ = (JaStyles, glass_tint)

rectangle((60, 18), width=56, height=18, style=card_style, text="api.internal:8443",
text_style=mono_label)
save()
```

`drawlib.fonts` ships universal and script-specific font families that download automatically on first use and cache locally (manageable offline via `uv run drawlib cache download --all`):

- **Universal & Western Families**: `Font` (Noto Sans/Serif covering Latin + CJK), `FontSansSerif` (Lato, Montserrat, Poppins, Oswald, Raleway), `FontSerif` (Merriweather, Playfair Display, Platypi), and `FontRoboto` (Roboto, Condensed, Slab, Serif).
- **Code & Tabular Metrics**: `FontMonoSpace` (`ROBOTO_MONO_REGULAR` , `SOURCECODEPRO_REGULAR` , `SOURCEHANCODEJP_REGULAR`) and `FontSourceCode` .
- **CJK, Regional & Custom Files**: `FontJapanese` , `FontChinese` (Simplified, Traditional, Hong Kong), `FontKorean` , `FontArabic` , `FontThai` , `FontBrahmic` , and `FontFile("_assets/custom.ttf")` for project-bundled `.ttf` / `.otf` assets.

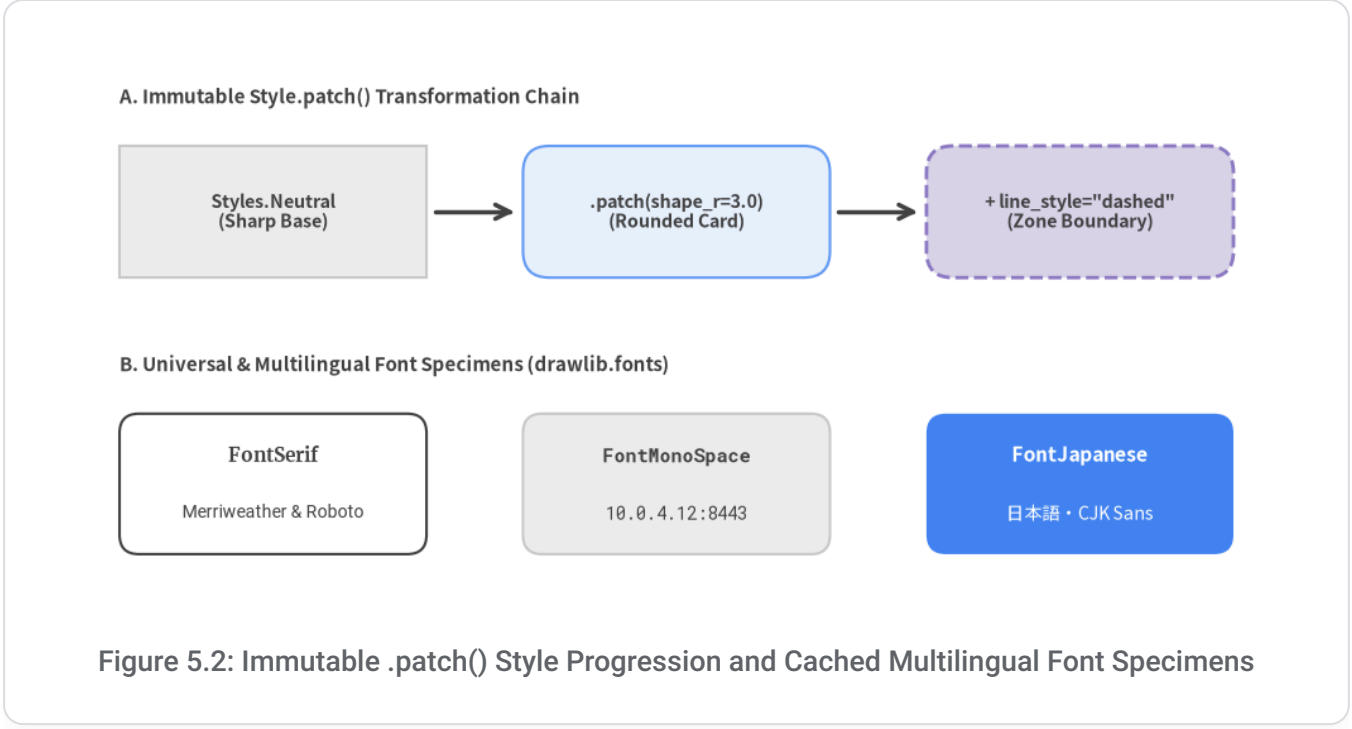


Figure 5.2: Immutable .patch() Style Progression and Cached Multilingual Font Specimens

6. 34 Icon Suites & Media Assets

Architecture blueprints and technical guides immediately gain clarity when services carry recognizable UI glyphs, official cloud icons, and crisp raster illustrations. Drawlib bundles **34 on-demand cached icon suites** in `drawlib.icons` alongside non-destructive raster processing and sub-canvas composition in `drawlib.images`.

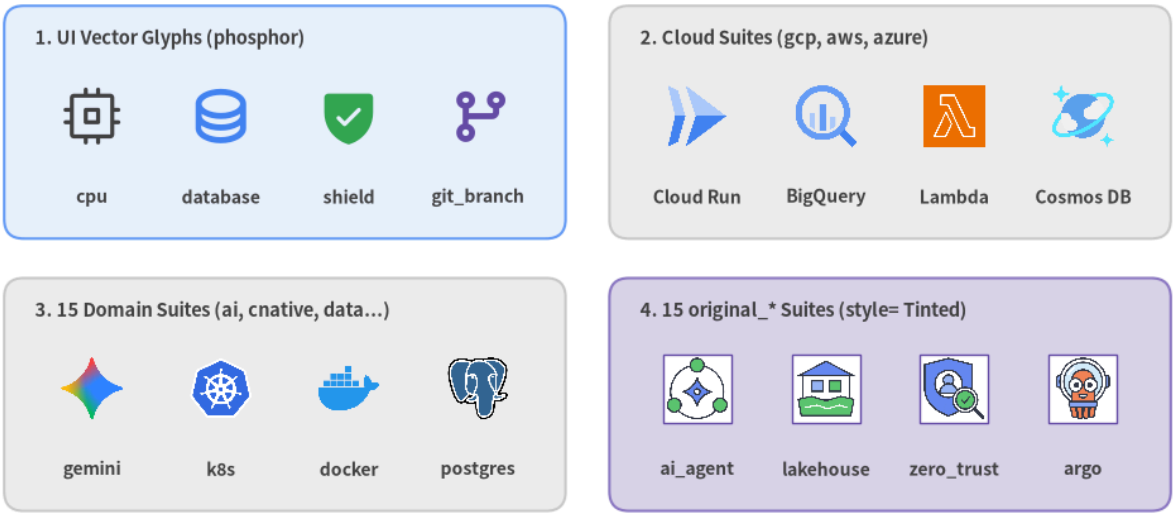


Figure 6.1: Representative Icons Across Drawlib's 34 Built-In UI, Cloud, Domain Brand, and Original 2D Suites

6.1 Overview of Drawlib's 34 Built-In Icon Suites

Every icon function in `drawlib.icons` follows the universal **center-anchored** `xy = (cx, cy)` contract with a `1:1` square `width=` and a mandatory keyword-only `*, style: Style` parameter. Binary icon packs download automatically on first reference and cache locally (`uv run drawlib cache download --all` pre-fetches all 34 suites for offline CI).

Tier	Suites (34 Total)	Rendering & `style=` Customization
1. General UI Vector Icons (1)	`phosphor` (1,500+ icons, plus low-level `font_icon`)	Crisp TrueType vector glyphs supporting 5 weights via `style.icon_style` (`"thin"`, `"light"`, `"regular"`, `"bold"`, `"fill"`—or `Styles.*Bold` / `*Flat`) and direct coloring via `style.icon_color`.
2. Cloud Provider Suites (3)	`gcp`, `aws`, `azure`	Official `512x512` RGBA service icons for Google Cloud, AWS, and Microsoft Azure architectures.
3. Multi-Brand Domain Suites (15)	`ai`, `business`, `client`, `cloud`, `cnative`, `code`, `data`, `devops`, `facility`, `itservice`, `network`, `security`, `server`, `social`, `workspace`	Curated multi-color vendor logos across AI/LLMs, CI/CD & DevOps, containers/Kubernetes (`cnative`), databases/streaming (`data`), frontend/mobile (`client`), hardware (`server`, `facility`), networking, and security.
4. `original_*` 2D Twin Suites (15)	`original_ai` through `original_workspace`	Unified flat 2D brand twins plus vendor-neutral concept icons (`original_ai.ai_agent`, `original_data.data_lakehouse`, `original_security.zero_trust`). Accepts `style=` background tinting (`image_tint_color`), framing (`image_border_*`), `alpha`, and `angle` (and monochrome recoloring via `Dimage.colorize()`).

6.2 Standalone Icon Calls vs. High-Level Enum Tokens

Drawlib provides two ergonomic ways to place icons depending on whether you are composing a custom canvas or declaring a structured diagram:

1. **Direct Canvas Calls (`drawlib.icons.<suite>.<name>`)**: Call `phosphor.cpu((25, 20), width=8.0, style=Styles.PrimaryBold)` (or `Styles.Accent.patch(icon_style="fill")`) and `gcp.cloud_run((60, 20), width=9.0, style=Styles.Primary)` to place an icon at exact canvas coordinates `(cx, cy)`.
2. **Typed Enum Tokens (`PhosphorIcon`, `GcpIcon`, `AwsIcon`, `CnativeIcon`, etc.)**: High-level diagram builders (`drawlib.diagrams.architecture`, `drawlib.diagrams.sequence`, and icon-aware `SmartArts`) export strongly typed `StrEnum` tokens so nodes automatically align icons alongside labels without manual coordinate math.

```python
from drawlib.canvas import save, setup
from drawlib.diagrams.architecture import ArchitectureDiagram, GcpIcon, Node,
PhosphorIcon
from drawlib.icons import phosphor
from drawlib.styles import Styles

setup(width=120, height=40)

# 1. Standalone direct icon call at exact center (cx, cy)
phosphor.sparkle((16, 20), width=7.5, style=Styles.Accent.patch(icon_style="fill"))

# 2. Enum tokens inside high-level ArchitectureDiagram nodes
arch = ArchitectureDiagram(
    node_style=Styles.PrimaryBold,
    node_text_style=Styles.DarkBold.patch(text_size=11.0),
    edge_style=Styles.DarkBold,
    edge_text_style=Styles.DarkBold.patch(text_size=10.0),
    node_card_style=Styles.Neutral,
)
gw = arch.add(Node("Edge Gateway", width=34, height=16, icon=PhosphorIcon.SHIELD_CHECK),
xy=(52, 20))
svc = arch.add(
    Node(
        "Cloud Run API",
        width=34,
        height=16,
        icon=GcpIcon.CLOUD_RUN,
        card_style=Styles.PrimaryNeutral,
    ),
    xy=(98, 20),
)
arch.connect(gw, svc, arrow="->")
arch.draw()

save()
```

6.3 Raster Assets (`image`), `Dimage` Pipelines & Nested Sub-Canvases

For screenshots, product photos, mascots, and composite sub-diagrams, `drawlib.images` provides three composable tools:

- `image(xy, width=..., image="_assets/linux.png", style=...)`: Places a raster file, `PIL.Image.Image`, or `Dimage` centered at `xy = (cx, cy)` while automatically preserving its aspect ratio. Always store project images under `_assets/` and reference relative paths

(`"_assets/linux.png"`). Use `style.patch(angle=..., alpha=..., image_tint_color=..., image_border_width=..., image_border_color=..., image_border_style=...)` to rotate, tint transparent backgrounds, or frame images cleanly.

- **Immutable `Dimage` Filter Chains**: Wrap any image in `Dimage("_assets/linux.png")` to chain non-destructive pixel operations—each returning a fresh `Dimage`:
  - **Geometry & Alpha Cleanup**: `.resize(width, height)`, `.crop(x, y, width, height)` *(where `(0, 0)` is the **bottom-left** pixel corner, matching Drawlib's Cartesian axes!)*, `.mirror()`, `.flip()`, `.trim()`, `.make_transparent(color="auto", tolerance=15)`, `.fill(color)`, and `.alpha(0.8)`.
  - **Color & Stylistic Filters**: `.grayscale()`, `.sepia()`, `.colorize(from_black_to=..., from_white_to=...)`, `.brightness(0.9)`, `.invert()`, `.posterize(num_colors=4)`, `.mosaic(block_size=16)`, `.blur()`, and `.line_extraction()`.
- **Nested Sub-Canvas Composition (`get_dimage_from_code`)**: Executes a self-contained Drawlib Python snippet in an isolated process and returns the rendered canvas in memory as a `Dimage`, making it easy to embed miniature badges or Picture-in-Picture diagrams inside larger layouts.

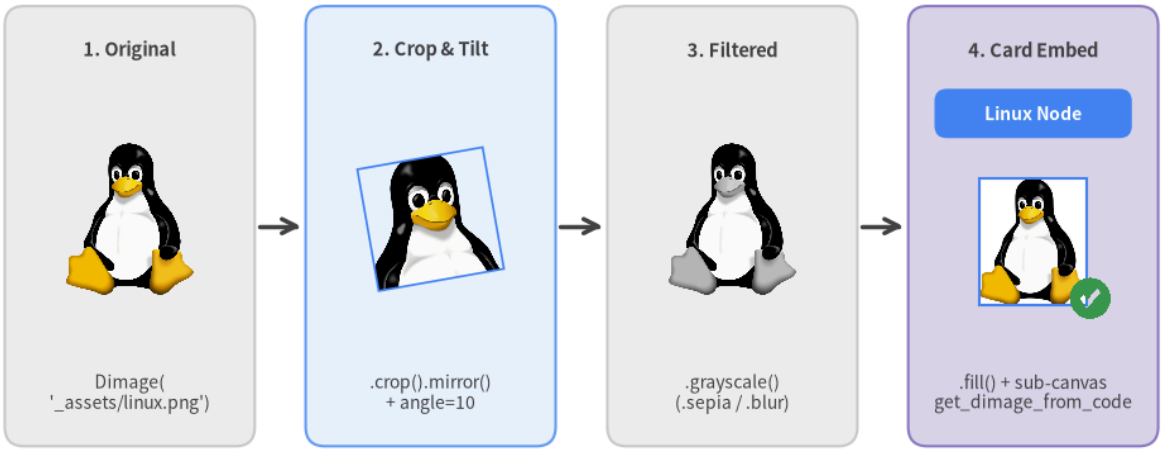


Figure 6.2: Non-Destructive Dimage("_assets/linux.png") Transformation Pipeline and Styled Card Embedding

7. SmartArts: 18 Structured Visual Components

Instead of calculating manual rectangle coordinates and text offsets for common engineering visuals, `drawlib.smartarts` provides **18 high-level components** that automate geometry, alignment, borders, and typography out of the box. Every SmartArt shares the same `GoogleStyles` design tokens and Cartesian canvas, letting you compose KPI executive headers, pipelines, SLA tables, and repository trees inside a single figure.

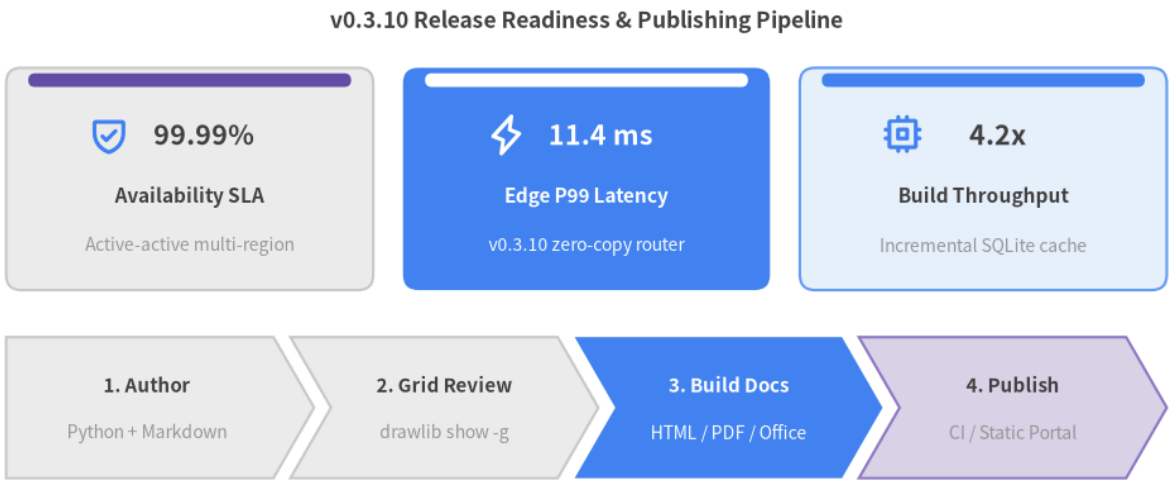


Figure 7.1: Executive Release Dashboard Combining KpiCards and ChevronProcess

7.1 Complete Catalog of 18 SmartArts & Coordinate Anchors

Because Drawlib uses a bottom-left `(0, 0)` Cartesian canvas where `+Y` points upward, knowing each SmartArt's default `.draw(xy=...)` coordinate anchor guarantees one-shot placement without trial-and-error shifts:

- **Bottom-Left** `(x0, y0)`: Most rectangular containers (`ChevronProcess`, `KpiCards`, `GeoMap`, `*Calendar`, etc.) start at `(x0, y0)` and extend rightward (`+X`) and upward

(`+Y`).

- **Top-Left** `(x0, y_top)`: Reading-order components (`Table`, `BulletPoints`, `SourceCode`, `TreeNode`) start at `(x0, y_top)` and grow downward (`-Y`), so always pass an upper `y_top` coordinate.
- **Center** `(cx, cy)`: Radial layouts (`Cycle`, `MindMapNode`) expand outward around `(cx, cy)`.

7.1.1 Processes, Cycles & Timelines

Component	Item / Helper Models	Default `.draw(xy=...)` Anchor	Primary Engineering Use Cases
`ChevronProcess`	`ChevronItem`	**Bottom-Left** (`align="bottom_left"`)	CI/CD pipelines, ETL workflows, phased rollouts (`flat_left_end=True`)
`Cycle`	`CycleItem`, `CycleCenter`	**Center** (`align="center"`)	PDCA feedback loops, OAuth token refresh cycles, SRE incident loops
`Roadmap`	`RoadmapItem`	**Bottom-Left** (`align="bottom_left"`)	Quarterly release milestones, migration timelines, postmortem chronicles

7.1.2 Cards, Grids & Summaries

Component	Item / Helper Models	Default `.draw(xy=...)` Anchor	Primary Engineering Use Cases
`KpiCards` (`KpiCard`)	`KpiCardItem`	**Bottom-Left** (`align="bottom_left"`)	Executive SLA metrics, latency targets, benchmark summary headers
`CardList`	`CardListItem`	**Bottom-Left** (`align="bottom_left"`)	Vertical or horizontal feature cards with optional `->` step connectors
`BoxList`	`BoxListItem`	**Bottom-Left** (`align="bottom_left"`)	Contiguous service strips, protocol frame headers, compact status blocks
`GridLayout`	`GridItem`	**Bottom-Left** (`align="bottom_left"`)	Multi-row/column spanning architecture matrices (`row=0` at bottom)
`Pyramid`	`PyramidItem`	**Bottom-Left** (`align="bottom_left"`)	Testing pyramids, storage/cache hierarchies, defense-in-depth layers
`Panel`	—	**Bottom-Left** (`align="bottom_left"`)	Titled container frames (`"filled"`, `"underline"`, `"floating"`, `"window"`)

7.1.3 Tables, Text & Code Blocks

Component	Item / Helper Models	Default `.draw(xy=...)` Anchor	Primary Engineering Use Cases
`Table`	—	**Top-Left** (`align="top_left"`)	Service SLA matrices, feature comparisons, quota schedules (`draw_flexible`)
`BulletPoints`	`BulletPointItem`	**Top-Left** *(fixed `(x, y)`)*	Architecture takeaways, design review checklists, slide callouts
`SourceCode`	`SourceCodeStyles` , `sourcecode`	**Top-Left** (`align="top_left"`)	Vector syntax-highlighted Python/YAML/JSON snippets inside diagrams

7.1.4 Hierarchies & Mind Maps

Component	Item / Helper Models	Default `.draw(xy=...)` Anchor	Primary Engineering Use Cases
`TreeNode`	—	**Top-Left** *(fixed `(x, y)`)*	Monorepo directory layouts, package hierarchies, CLI command trees
`MindMapNode`	—	**Center** *(fixed `(cx, cy)`)*	4-way radial system taxonomies, domain decompositions, org charts

7.1.5 Geographic & Temporal

Component	Item / Helper Models	Default `.draw(xy=...)` Anchor	Primary Engineering Use Cases
`GeoMap`	—	**Bottom-Left** (`align="bottom_left"`)	Multi-region cloud deployments (`World` , `Countries` , `Cities` , `lonlat_to_xy`)
`MonthCalendar`	`CalendarItem`	**Bottom-Left** (`align="bottom_left"`)	7-column monthly or multi-week release schedules (`"YYYY-MM-DD"`)
`WeekCalendar`	`CalendarItem`	**Bottom-Left** (`align="bottom_left"`)	N-day hourly sprint and maintenance windows (`"YYYY-MM-DD HH:MM"`)
`DayCalendar`	`CalendarItem`	**Bottom-Left** (`align="bottom_left"`)	Single-day incident timelines and minute-level change runbooks

> **Tip — Aligning Top-Left and Bottom-Left Components Side by Side**: To align a `Table` or `TreeNode` of height `H` with a Bottom-Left box sitting at `(x0, y_bottom)`, pass `xy=(x0, y_bottom + H)` as the Top-Left anchor (or pass `align="bottom_left"` directly on `Table` and `SourceCode`).

7.2 Top-Left Composition: Pairing Table and TreeNode

`Table` and `TreeNode` both default to **Top-Left** `(x0, y_top)` coordinates, making them natural partners for side-by-side specification figures:

- `Table` **Styling Order Rule**: `Table` evaluates cell rules in registration order. Because `0` is an even row index, always call `set_style_cell_evenodd(...)` **before** `set_style_cell_header(...)` so the header row (`row 0`) retains its dark/accent banner, then highlight individual target cells last via `set_style_cell(..., rows=[r], columns=[c])`.
- `TreeNode` **Cascading & Icons**: Define all five layout attributes (`text_style`, `line_style`, `line_horizontal_margin`, `line_horizontal_length`, `line_vertical_margin`) on the root `TreeNode`—descendants inherit them automatically. Attach vector `phosphor` icons via `TreeNode.register_drawing_item(...)` and `.set_drawing_item(name)`. Pass pre-built `TreeNode` children via `children=[...]`, or append string labels dynamically via `node.add("filename.py")`.

Archetype Output Matrix (Table)

Archetype	Primary Output	Office / Print
doc	Chapter HTML	PDF + DOCX
site	Multi-Page Docs	llms.txt + SEO
catalog	Searchable Ref	JSON Index
slide	16:9 Web Deck	PDF + PPTX

Scaffolded Project Tree (TreeNode)

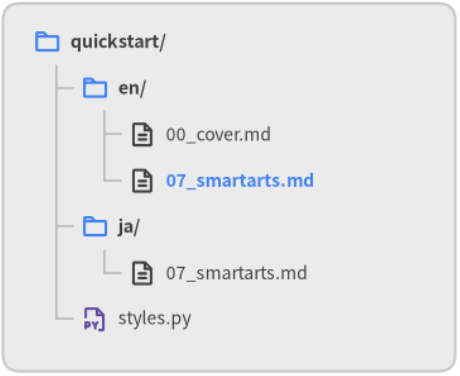


Figure 7.2: Side-by-Side Top-Left Composition Pairing a Styled Table and Project TreeNode

7.3 Step-by-Step Slide & Animation Builds with `show=False`

Every stateful SmartArt follows a four-phase lifecycle (**Instantiate -> Register via `.add()` -> Mutate `.items` / `.children` -> Render via `.draw()`**). Each registered item or node exposes a boolean `show: bool = True` flag (`ChevronItem`, `KpiCardItem`, `CardListItem`, `BoxListItem`, `CycleItem`, `GridItem`, `PyramidItem`, `RoadmapItem`, `BulletPointItem`, `CalendarItem`, `TreeNode`, and `MindMapNode`).

Setting `item.show = False` (or passing `show=False` in `.add(...)`) hides that specific step while **preserving the exact bounding widths, column positions, orbit angles, and vertical row spacing of all sibling items**. This makes progressive slide builds and multi-frame `drawlib.anim` animations effortless—earlier steps never jump or stretch as later steps appear:

```python
from drawlib.canvas import save, setup
from drawlib.smartarts import ChevronProcess
from drawlib.styles import Styles

setup(width=120, height=32)

flow = ChevronProcess(
    style=Styles.Neutral,
    text_style=Styles.DarkBold.patch(text_size=10.5),
    description_style=Styles.Muted.patch(text_size=10.0),
    flat_left_end=True,
)
flow.add("1. Ingest", description="Parquet Stream")
flow.add(
    "2. Validate",
    description="Schema Check",
    style=Styles.PrimaryFlat,
    text_style=Styles.WhiteBold.patch(text_size=10.5),
    description_style=Styles.White.patch(text_size=10.0),
)
# Hidden in Frame 1, yet Steps 1 & 2 keep their exact 4-column widths!
step3 = flow.add("3. Enrich", description="Vector Join", show=False)
step4 = flow.add("4. Serve", description="Low-Latency API", show=False)

flow.draw(xy=(8.0, 8.0), width=104.0, height=16.0)
save()
```

Query any SmartArt's full constructor reference on demand via `uv run drawlib rules show lib-smartarts-overview` (or component manuals such as `lib-smartarts-calendar`, `lib-smartarts-geomap`, and `lib-smartarts-source-code`).

8. Quantitative Charts: 10 Pure-Python Chart Types

Technical whitepapers, benchmark reports, and architecture reviews rarely stop at box-and-arrow diagrams—they need rigorous quantitative evidence. `drawlib.charts` provides **10 pure-Python chart types** built natively on Drawlib's rendering engine, all anchored at **Bottom-Left** `(x0, y0)` (`align="bottom_left"`) so charts sit alongside `SmartArts`, icons, and architecture callouts on a single unified canvas.

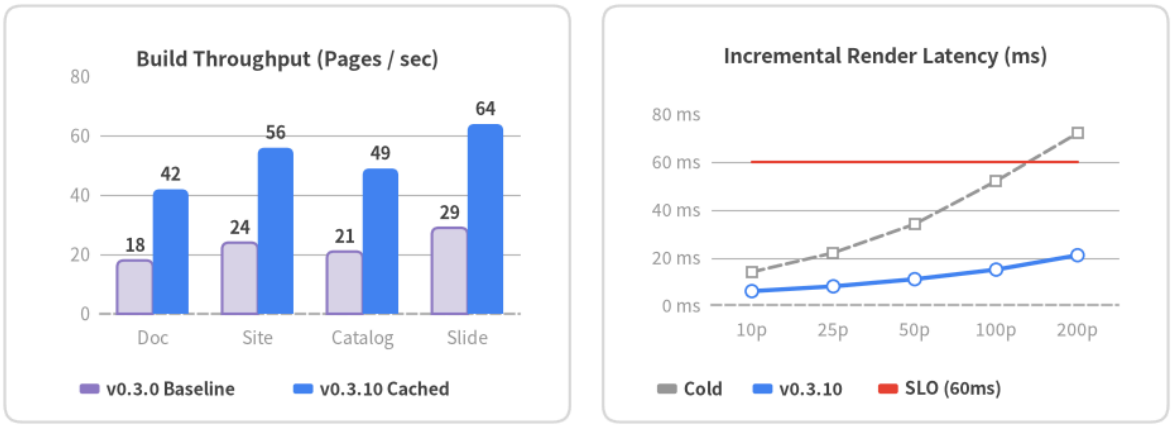


Figure 8.1: Side-by-Side Quantitative Dashboard — Grouped BarChart Throughput and LineChart Tail Latency

8.1 Why `drawlib.charts` Beats Matplotlib in Technical Documentation

Embedding external Matplotlib scripts into multi-language engineering docs introduces mismatched fonts, brittle `rcParams` boilerplate, and isolated image files that cannot interact with surrounding diagrams. `drawlib.charts` solves all four friction points natively:

- **Identical Cartesian Coordinate System**: Every chart draws directly at `xy=(x0, y0)` on the active Drawlib canvas, making multi-chart dashboards, framing cards, and `KpiCards`

headers effortless to align on a single grid.

- **Unified `GoogleStyles` Tokens & CJK Fonts**: Charts inherit `Styles`, `Colors`, and locale-specific fonts (`ja/styles.py`) automatically—eliminating missing glyph boxes (□□□) and clashing color palettes.
- **Presence-Based Styling & Collision-Free Legends**: Leaving optional parameters (`grid_style`, `value_text_style`, `background_style`) as `None` cleanly omits visual clutter, while decoupled `chart.draw_legend(xy=...)` and `chart.draw_colorbar(xy=...)` calls let you place legends with zero plot collisions.
- **Layout-Preserving Animation (`show` & `draw_ratio`)**: Mutating `series.show = False` or sweeping `series.draw_ratio` from `0.0` to `1.0` animates bars, curves, wedges, and Gantt tasks while locking the full 100% dataset's axis bounds in place.

8.2 Complete Catalog of All 10 Chart Types

All **10 chart classes** across the 9 `drawlib.charts.*` submodules default to **Bottom-Left** `(x0, y0)` (`align="bottom_left"`, with optional `align="center"` and `scale: float = 1.0`):

#	Chart Class	Submodule	Default `.draw(xy=...)` Anchor	Core Modes & Capabilities
1	`BarChart`	`drawlib.charts.bar`	**Bottom-Left** `(x0, y0)`	`"vertical"` / `"horizontal"`, `"group"` / `"stack"`, `bar_r`, value labels, `"log"` scale
2	`BoxChart`	`drawlib.charts.box`	**Bottom-Left** `(x0, y0)`	Raw sample lists or `BoxStats`, `"iqr"` / `"percentile"` whiskers, `show_points` strip jitter
3	`LineChart`	`drawlib.charts.line`	**Bottom-Left** `(x0, y0)`	Multi-series polylines or `smooth=True` splines, vertex markers, `point_shape="none"` thresholds
4	`AreaChart`	`drawlib.charts.area`	**Bottom-Left** `(x0, y0)`	Filled `"overlap"` or cumulative `"stack"` trend curves with configurable `fill_alpha`
5	`PieChart`	`drawlib.charts.pie`	**Bottom-Left** `(x0, y0)`	Solid pie (`hole_ratio=0.0`) & donut (`hole_ratio=0.5..0.7`) with `center_text` KPI badge
6	`RadarChart`	`drawlib.charts.radar`	**Bottom-Left** `(x0, y0)`	Multi-spoke `"polygon"` or `"circle"` spider capability matrices (`levels`, `min_value` .. `max_value`)

#	Chart Class	Submodule	Default `.draw(xy=...)` Anchor	Core Modes & Capabilities
7	`ScatterChart`	`drawlib.charts.scatter`	**Bottom-Left** `(x0, y0)`	2D `(x, y)` correlation plots and variable-radius bubble sizing via `Point(x, y, radius=...)`
8	`GanttChart`	`drawlib.charts.gantt`	**Bottom-Left** `(x0, y0)`	Project timelines with `Section`, `Task` progress bars, `Milestone`, `Marker`, and `Dependency`
9	`HeatmapChart`	`drawlib.charts.heatmap`	**Bottom-Left** `(x0, y0)`	2D intensity matrices (`min_style`..`max_style` or `styles`), `auto_contrast_text`, `draw_colorbar`
10	`HeatmapOverlay`	`drawlib.charts.heatmap`	**Bottom-Left** `(x0, y0)`	Spatial density `Spot` fields (`mode="smooth"` or `"contour"`) layered over maps or floorplans

8.2.1 Categorical, Statistical & Matrix Charts

- `BarChart` & `BoxChart`: Share orientation-aware `value_axis` and `category_axis` helpers (`configure_x_axis(...)` / `configure_y_axis(...)`) supporting Nice Numbers linear ticks and base-10 logarithmic (`scale="log"`) axes for wide-dynamic-range benchmarks.
- `HeatmapChart` & `HeatmapOverlay`: Map scalar intensities across color stops (`min_style=Styles.White, max_style=Styles.PrimaryFlat` or multi-stop `styles=[...]`). `HeatmapChart` automatically flips cell label contrast (`auto_contrast_text=True`), while `HeatmapOverlay` renders radial `Spot(x, y, value, radius=...)` fields directly over `GeoMap` or architecture canvases.

8.2.2 Continuous, Radial & Temporal Charts

- `LineChart`, `AreaChart` & `ScatterChart`: Plot continuous trajectories, stacked resource envelopes, and multidimensional `(x, y, radius)` correlations. Setting `point_shape="none"` with `line_style="dotted"` or `"dashdot"` on a `LineChart` series creates clean SLO/SLA reference lines without vertex clutter.
- `PieChart`, `RadarChart` & `GanttChart`: Cover part-to-whole ratios (`hole_ratio > 0.0` unlocks donut rings with multiline `center_text` badges and automatic `< 4%` slice label suppression), multi-axis architecture trade-offs (`RadarChart`), and milestone execution schedules (`GanttChart`).

8.3 Statistical Storytelling with `BoxChart` & Progressive Animations

Effective quantitative figures follow strict **data storytelling discipline**: never assign five saturated rainbow colors to five competing series. Ground historical baselines, control groups, and secondary tiers in calm neutral tokens (`Styles.Neutral` , `Styles.PrimaryNeutral` , `Styles.SecondaryNeutral` , or dashed `Styles.Muted`), and reserve `Styles.PrimaryFlat` or `Styles.AccentFlat` exclusively for the target series under evaluation.

While bar charts hide tail spikes behind averages, `BoxChart` exposes medians, interquartile spreads (`Q1` – `Q3`), whiskers, and p99 outliers simultaneously—accepting either raw sample sequences (`list[float]`) or pre-aggregated telemetry (`BoxStats`) with automatic high-contrast median strokes:

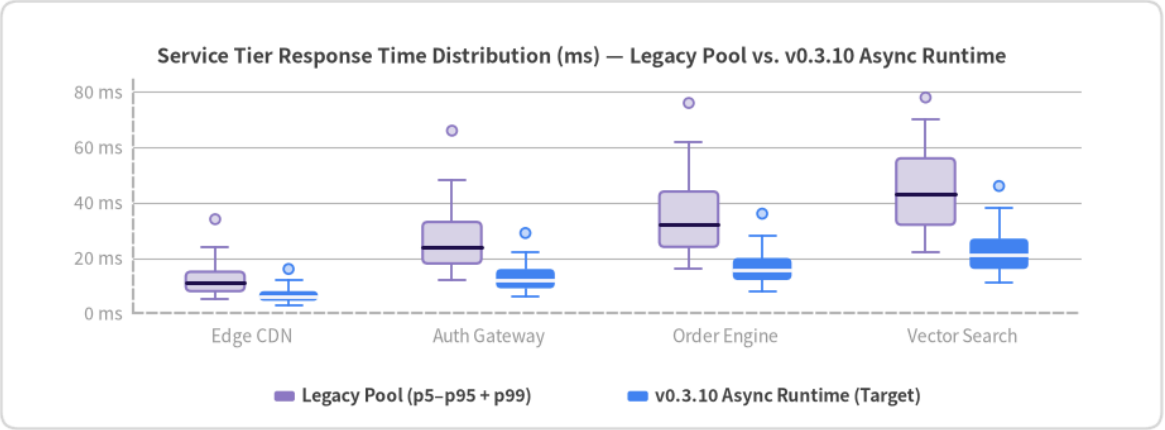


Figure 8.2: Tail-Latency Distribution Across Service Tiers Using BoxChart and BoxStats

---

When working with small benchmark runs (`N = 8..25`), pass raw `list[float]` samples with `show_box=True, show_points=True, jitter=0.42` to overlay deterministic strip points on top of each quartile box—or sweep `draw_ratio` (`0.0` to `1.0`) on `LineChart` / `BarChart` series to render smooth multi-frame `drawlib.anim` builds:

```python
from drawlib.canvas import save, setup
from drawlib.charts.line import LineChart
from drawlib.styles import Styles

setup(width=110, height=56)

trend = LineChart(
    axis_line_style=Styles.MutedDashed,
    categories=["T1", "T2", "T3", "T4", "T5"],
    width=90.0,
    height=38.0,
    smooth=True,
    axis_text_style=Styles.Muted.patch(text_size=10.0),
    grid_style=Styles.MutedThin,
    title="Progressive Rollout Throughput (60% Keyframe)",
    title_style=Styles.DarkBold.patch(text_size=11.5),
)
trend.add_series("Baseline", [40, 42, 41, 43, 42], style=Styles.SecondaryNeutral,
line_style="dashed")
# Renders 60% of the curve left-to-right while keeping full T1..T5 axis bounds fixed!
focal = trend.add_series("v0.3.10 Canary", [42, 58, 74, 89, 96],
style=Styles.PrimaryFlat, draw_ratio=0.6)

trend.configure_y_axis(min_value=0, max_value=100, tick_step=25, unit="k rps")
trend.draw(xy=(10.0, 10.0))
save()
```

Inspect detailed axis, legend, colorbar, and constructor options for any chart family via `uv run drawlib rules show lib-charts-overview` (or `lib-charts-bar`, `lib-charts-box`, `lib-charts-line`, `lib-charts-pie`, `lib-charts-gantt`, and `lib-charts-heatmap`).

9. Cloud & System Architecture Diagrams

Distributed systems are defined by trust boundaries, network topologies, and service dependencies. `drawlib.diagrams.architecture` turns cloud infrastructure blueprints into clean, version-controlled Python code—combining hierarchical `NodeGroup` containers, 33 built-in vendor & vector icon enums (`GcpIcon` , `AwsIcon` , `AzureIcon` , `PhosphorIcon`), smart boundary edge routing, and spaghetti-free `junction()` buses.

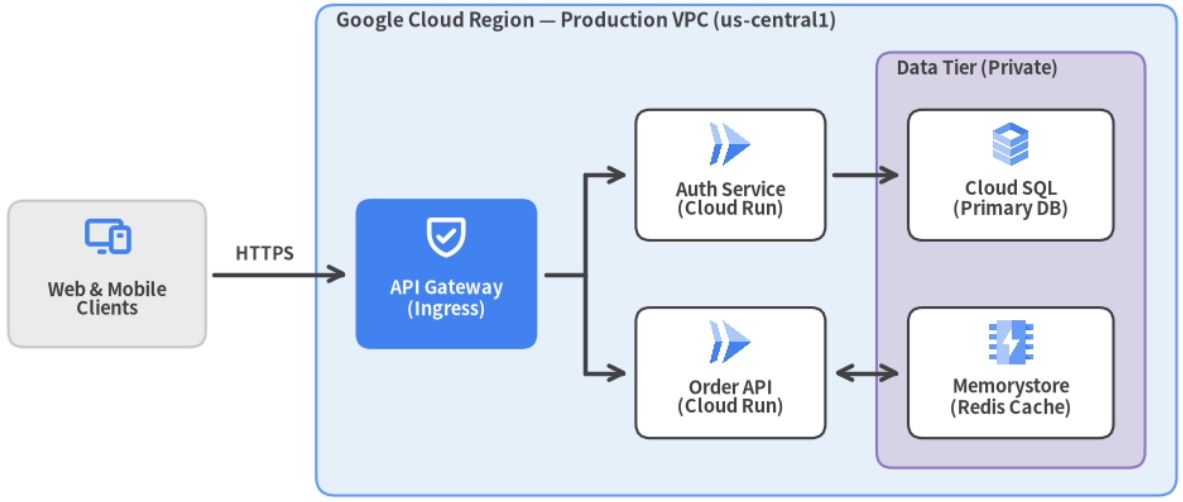


Figure 9.1: Production Cloud VPC Topology with External Client, Focal API Gateway, Junction Fan-Out, and Data Tier

9.1 `ArchitectureDiagram` , `Node` & Coordinate Semantics

`ArchitectureDiagram` acts as the top-level container and establishes a **three-tier style cascade** (*Element Override -> Diagram Default -> Built-in Fallback*). Its four required keyword-only defaults— `node_style` , `node_text_style` , `edge_style` , and `edge_text_style` —plus optional `node_card_style` keep typography and stroke weights uniform across dozens of microservices without repeating style kwargs on every node.

Understanding how `d.add(..., xy=...)` and `group.add(..., xy=...)` interpret `xy` prevents off-by-half-width placement bugs:

Registration Call	Anchor Point of `xy`	Coordinate Frame	Global Diagram Position Formula
`d.add(Node(...), xy=(cx, cy))`	**Center** `(cx, cy)`	Global diagram space	`(cx, cy)`
`d.add(NodeGroup(...), xy=(x0, y0))`	**Bottom-Left** `(x0, y0)`	Global diagram space	Bottom-Left at `(x0, y0)`
`group.add(Node(...), xy=(lx, ly))`	**Center** `(lx, ly)`	**Local** to `group` `(0, 0)`	`(x0 + lx, y0 + ly)`
`group.add(NodeGroup(...), xy=(lx0, ly0))`	**Bottom-Left** `(lx0, ly0)`	**Local** to `group` `(0, 0)`	Bottom-Left at `(x0 + lx0, y0 + ly0)`
`d.junction((jx, jy))`	**Exact Point** `(jx, jy)`	Global diagram space	`(jx, jy)`

9.1.1 Pairing Official Cloud & Monochrome Icons with Card Contrast

- **Multi-Color Cloud & Domain Icons (** `GcpIcon` **,** `AwsIcon` **,** `AzureIcon` **,** `DataIcon` **,** `CnativeIcon` **, etc.)**: Render `512x512` official RGBA artwork. Place them on `Styles.White` or light `Styles.Neutral` cards (`card_style=Styles.White.patch(shape_r=1.8)`) so brand colors stay crisp and legible.
- **Focal Hero Nodes (** `Styles.PrimaryFlat` **/** `Styles.AccentFlat` **)**: Reserve saturated card fills for 1–2 architectural focal points (such as an Ingress Gateway or Event Router), and pair them with monochrome `PhosphorIcon` glyphs tinted via `style=Styles.WhiteBold` and `text_style=Styles.WhiteBold`.

9.2 Network Boundaries with `NodeGroup` & External Actor Rule

`NodeGroup` models hierarchical boundaries—Cloud Regions, VPCs, Availability Zones, Kubernetes Clusters, Public/Private Subnets, and Data Tiers. Groups can either **auto-size** around their children via `padding: float = 5.0` (when `width=None, height=None`) or lock to explicit `width` and `height` dimensions for crisp multi-tier grid alignment.

> **Architectural Boundary Rule — Keep External Actors Outside the Cloud `NodeGroup`**: Never place end users, mobile apps, browsers, or third-party SaaS webhooks *inside* your Cloud Region or VPC `NodeGroup`. Register external actors in global diagram space via `d.add(Node(...), xy=...)` on the left or top perimeter, and route them across the `NodeGroup` border into an explicit edge/ingress node (CDN, Load Balancer, or API Gateway).

9.3 Edge Routing, Bidirectional Arrows & `junction()` Fan-Out Buses

`d.connect(start, end, ...)` (as well as `node.connect(...)` and `Edge(...)`) connects any `Node`, `NodeGroup`, or `Junction` while automatically clipping arrows cleanly at outer bounding boxes:

- **Three Routing Modes (`routing`)**:
  - `"orthogonal"` *(default)*: Right-angled Manhattan L-bends and Z-bends between service tiers.
  - `"direct"`: Straight Euclidean segments for aligned horizontal/vertical flows or bus trunks.
  - `"curved"`: Smooth curved arcs/splines (steerable via `.via((x, y))`) ideal for async callbacks or replication paths.
- **Boundary Attachment, Padding & Arrows**: Endpoints automatically select the closest boundary side toward the target or first `.via((x, y))` waypoint. Use `padding=1.0` (or asymmetric `padding=(1.0, 0.0)`) for crisp breathing room, `arrow="<->"` for bidirectional read/write cache links, and `xy_shift=(dx, dy)` on `edge_text_style` so labels never collide with line strokes.
- **Spaghetti-Free Fan-Out Buses (`d.junction(xy)`, `node.fork()`, `edge.add_point()`)**: Instead of drawing N diagonal lines from a gateway to N downstream workers, route a single `"direct"` trunk (`arrow="-"`) into shared `d.junction((x, y))` waypoints (or call `gateway.fork([svc1, svc2], at_x=...)`).

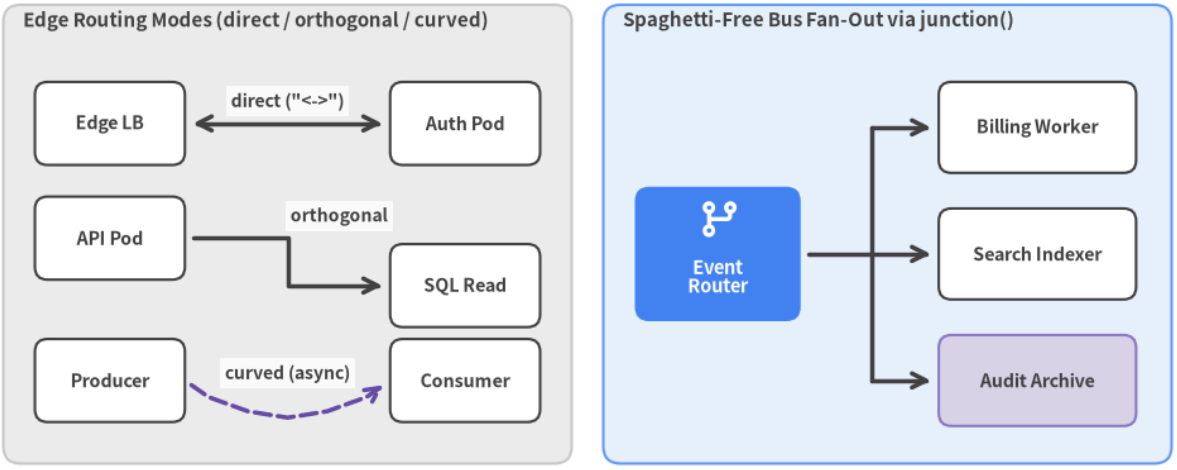


Figure 9.2: Clean Connection Patterns — Direct, Orthogonal, Curved Edges, and a Junction Fan-Out Bus

---

For standard 1-to-N orthogonal fan-outs or dead-letter stream taps, `node.fork(targets, at_x=...)` and `edge.add_point((x, y))` create and wire `Junction` waypoints in a single method call:

```python
from drawlib.canvas import save, setup
from drawlib.diagrams.architecture import ArchitectureDiagram, GcpIcon, Node,
PhosphorIcon
from drawlib.styles import Styles

setup(width=124, height=54)

d = ArchitectureDiagram(
    node_style=Styles.PrimaryBold,
    node_text_style=Styles.DarkBold.patch(text_size=10.2),
    edge_style=Styles.DarkBold,
    edge_text_style=Styles.DarkBold.patch(text_size=10.0),
    node_card_style=Styles.Neutral,
)

lb = d.add(
    Node(
        "Ingress LB",
        width=24.0,
        height=16.0,
        icon=PhosphorIcon.SHIELD_CHECK,
        icon_size=5.8,
        style=Styles.WhiteBold,
        text_style=Styles.WhiteBold.patch(text_size=10.5),
        card_style=Styles.PrimaryFlat.patch(shape_r=1.8),
    ),
    xy=(20.0, 25.0),
)
pod_a = d.add(Node("Worker A", width=24.0, height=15.0, icon=GcpIcon.CLOUD_RUN,
icon_size=5.8), xy=(84.0, 36.0))
pod_b = d.add(Node("Worker B", width=24.0, height=15.0, icon=GcpIcon.CLOUD_RUN,
icon_size=5.8), xy=(84.0, 14.0))

# Auto-creates a shared Junction at x=52.0 and fans out cleanly to both pods
lb.fork([pod_a, pod_b], at_x=52.0, padding=1.2)

d.draw(xy=(6.0, 2.0))
save()
```

Explore all 33 architecture icon enums and advanced waypoint APIs via `uv run drawlib rules show lib-diagrams-architecture` and `uv run drawlib rules show lib-icons-overview`.

10. Software Design Diagrams: Sequence, Flow, State, Class & ER

Beyond cloud infrastructure topologies (`ArchitectureDiagram`), `drawlib.diagrams` provides five specialized software engineering engines— `SequenceDiagram`, `FlowDiagram`, `StateDiagram`, `ClassDiagram`, and `ERDiagram` —that render standard UML 2.x, ISO 5807, and Crow's Foot models directly from declarative Python code.

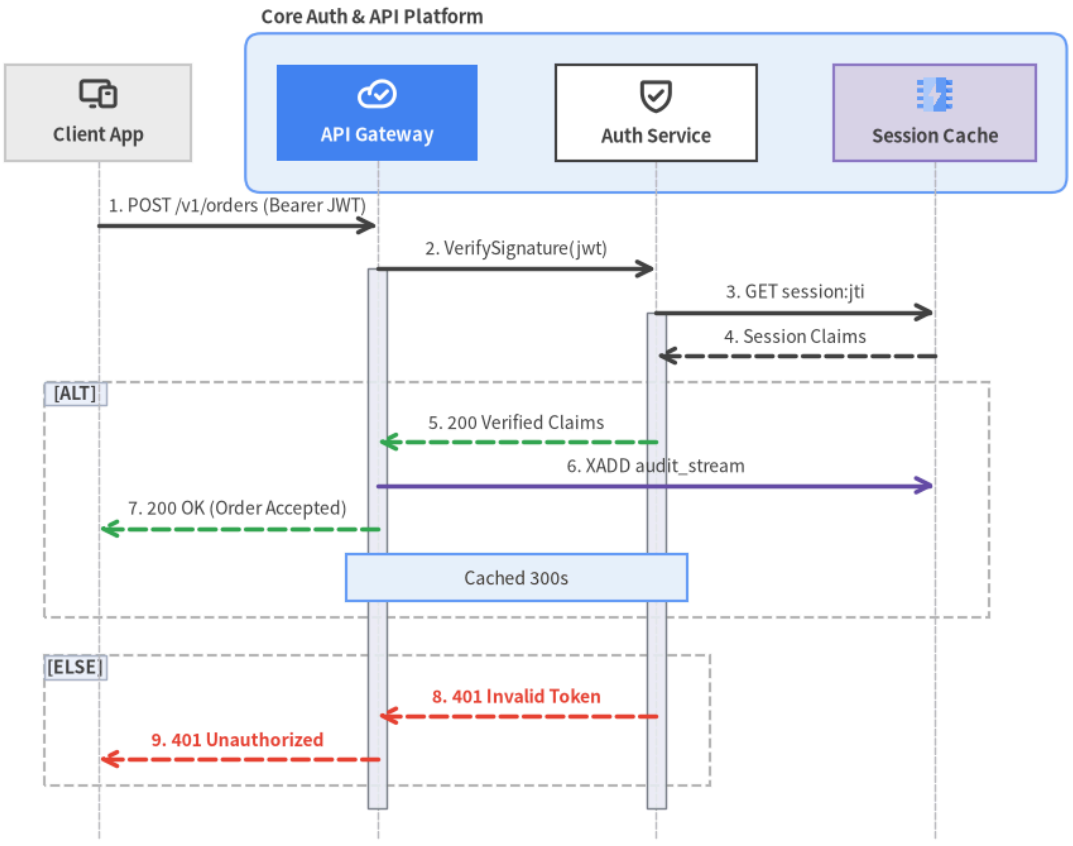


Figure 10.1: OAuth 2.0 & Token Verification RPC Sequence with ParticipantGroup, Activations, and Alt/Else Frame

10.1 Overview of the Five Software Modeling Engines

Each diagram engine in `drawlib.diagrams` shares a consistent builder lifecycle: configure base styles on the diagram constructor, register domain elements via `.add()`, connect them semantically, query `.get_size()` when exact bounding dimensions are needed, and render onto the canvas with `.draw(xy=(x0, y0))` (bottom-left anchored by default).

Module	Primary Class	Core Elements	Coordinate Strategy	Primary Use Cases
`drawlib.diagrams.sequence`	`SequenceDiagram`	`Participant`, `ParticipantGroup`, `Message`, `Note`, `Block`	Auto-advances `col_width` (X) and `step_y` (Y)	RPC/REST call timelines, OAuth handshakes, async messaging
`drawlib.diagrams.flow`	`FlowDiagram`	`Start`, `End`, `Process`, `Decision`, `Data`, `Lane`, `Junction`	Center `xy=(cx, cy)` across shared global/swimlane space	CI/CD gates, business workflows, cross-team swimlanes
`drawlib.diagrams.state`	`StateDiagram`	`State`, `InitialState`, `FinalState`, `ChoiceState`, `ForkJoinState`	Center `xy=(cx, cy)` with curved/orthogonal transitions	Protocol FSMs, worker lifecycles, concurrent statecharts
`drawlib.diagrams.class_diagram`	`ClassDiagram`	`ClassNode`, `AttributeInfo`, `MethodInfo`, `ClassRelationship`	Center `xy=(cx, cy)` with auto-sized 3-compartment boxes	OOP domain models, interface contracts, package hierarchies
`drawlib.diagrams.er`	`ERDiagram`	`Entity`, `ColumnInfo`, `Relationship`	Center `xy=(cx, cy)` with row-accurate `start_column` / `end_column`	Relational SQL schemas, PK/FK constraints, Crow's Foot models

10.2 API Interactions with SequenceDiagram

`SequenceDiagram` (`from drawlib.diagrams.sequence import SequenceDiagram, Participant, ParticipantGroup`) eliminates manual vertical coordinate math for chronological protocols:

- **Participants, Groups & Icons**: Register endpoints left-to-right with `d.add(Participant(...))` or group backend services inside `ParticipantGroup(title=..., padding=4.0)`. All 33 icon enums (`PhosphorIcon`, `GcpIcon`, `AwsIcon`, `AzureIcon`, etc.) are re-exported directly from `drawlib.diagrams.sequence`.
- **Sync, Async & Return Messages**: Call `a.request(b, "label")` for solid synchronous calls, pass `is_async=True` for open-arrow asynchronous events, call `b.reply(a, "label")` for dashed return arrows, or call `a.request(a, "Validate")` for self-invocations. Set `autonumber=True` on `SequenceDiagram` to prefix steps automatically (`1.`, `2.`, ...).
- **Lifeline Activations & Notes**: Bracket active execution spans with `p.activate()` and `p.deactivate()`, and attach timeline annotations via `p.note("text", pos="right")` or spanning `d.note("text", over=[p1, p2])`.
- **Combined Fragments (`alt`, `else_`, `loop`, `opt`, `par`)**: Wrap conditional or repetitive flows in Python `with` blocks (`with d.alt("Guard"):` / `with d.else_("Fallback"):`, `with d.loop("Every 30s"):`, `with d.opt("2FA"):`, `with d.par("Fan-out"):`).
- **Zero-Waste Canvas Sizing**: Query `w, h = d.get_size()` after recording messages and pass integer bounds `setup(width=math.ceil(w) + 8, height=math.ceil(h) + 8)` before `d.draw(xy=(4.0, 4.0))` so the canvas wraps the lifelines tightly without blank half-canvas margins.

10.3 Workflows & State Lifecycles: FlowDiagram & StateDiagram

`FlowDiagram` models operational decisions and swimlanes, while `StateDiagram` models event-driven object lifecycles and finite state machines:

- `FlowDiagram` (`drawlib.diagrams.flow`):
  - **ISO 5807 Symbols**: `Start` and `End` (stadium terminals), `Process` (action rectangle), `Decision` (branching diamond), and `Data` (I/O parallelogram), plus `Junction` (`flow.junction(xy)` or `node.fork([...])`) for clean wire merges.
  - **Swimlanes (`flow.add_lane`)**: Add vertical columns (`lane_orientation="vertical"`) or horizontal rows (`"horizontal"`). All lanes share a single global coordinate plane, so assigning the same `y` coordinate aligns cross-lane steps effortlessly.
  - **High-Contrast Node Styling**: Ground standard steps in `Styles.Neutral` or `Styles.PrimaryNeutral` with `Styles.DarkBold` text. When highlighting a focal step with `Styles.PrimaryFlat` or `Styles.AccentFlat`, **always set** `text_style=Styles.WhiteBold`—never leave dark or blue `PrimaryBold` text on a dark `PrimaryFlat` fill.
- `StateDiagram` (`drawlib.diagrams.state`):
  - **State & Pseudo-State Classes**: Use `State(name, entry=..., do=..., exit=...)` (plus fluent `.add_action(kind, action)` for `StateAction` rows), `InitialState()`, `FinalState()`, `ChoiceState()` (guard diamond), and `ForkJoinState(orientation="horizontal" | "vertical")` (concurrency bar).
  - **UML Transitions (`sd.connect`)**: Pass `event="...", guard="...", action="..."` to format `"event [guard] / action"` labels automatically, use `bend=0.25` with `routing="curved"` to separate bidirectional arcs, and connect a state to itself (`sd.connect(s, s, side="top")`) for self-transitions.

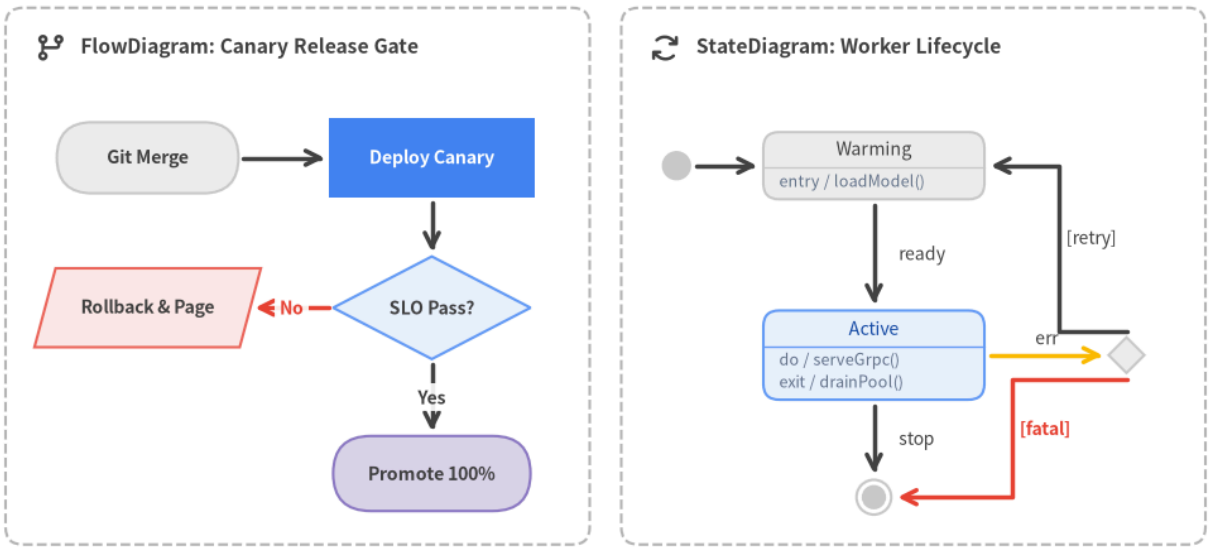


Figure 10.2: Side-by-Side Software Modeling — CI/CD Canary FlowDiagram (Left) and Worker Lifecycle StateDiagram (Right)

10.4 Object-Oriented Models with ClassDiagram

`ClassDiagram` (`drawlib.diagrams.class_diagram`) builds three-compartment UML 2.0 class boxes (`ClassNode`) whose heights expand automatically to fit their stereotypes (`stereotype="interface"` or `is_abstract=True`), attributes, and methods:

- **Fluent Member Registration**: Chain `.add_attribute(name, type, visibility="-", default_value=..., is_static=...)` and `.add_method(name, params, return_type, visibility="+", is_abstract=...)` with standard UML visibility markers (`"+"` public, `"-"` private, `"#"` protected, `"~"` package).
- **Six Canonical UML Relationships**: Call `cd.connect(source, target, relationship_type=...)` with `"inheritance"` (solid line, hollow triangle), `"realization"` (dashed line, hollow triangle), `"composition"` (solid diamond at owner), `"aggregation"` (hollow diamond), `"dependency"` (dashed open arrow), or `"association"` (solid link), complete with `start_multiplicity`, `end_multiplicity`, and `end_role`.

```
from drawlib.canvas import save, setup
from drawlib.diagrams.class_diagram import ClassDiagram, ClassNode
from drawlib.fonts import FontRoboto, FontSourceCode
from drawlib.styles import Styles

setup(width=140, height=64)

cd = ClassDiagram(
    node_style=Styles.Neutral.patch(text_font=FontSourceCode.ROBOTO_MONO,
text_size=10.0),
    header_style=Styles.PrimaryNeutral.patch(text_font=FontRoboto.ROBOTO_BOLD,
text_size=10.8),
    edge_style=Styles.DarkBold,
    edge_text_style=Styles.Dark.patch(text_size=10.0),
)

repo = cd.add(
    ClassNode("PaymentGateway", stereotype="interface", width=44,
header_style=Styles.SecondaryNeutral)
    .add_method("authorize", "amount: Decimal", "AuthReceipt"),
    xy=(32, 42),
)
stripe_gw = cd.add(
    ClassNode("StripeGateway", width=44)
    .add_attribute("api_client", "HttpClient", visibility="-")
    .add_method("authorize", "amount: Decimal", "AuthReceipt"),
    xy=(32, 16),
)
invoice = cd.add(
    ClassNode(
        "Invoice",
        width=40,
        header_style=Styles.PrimaryFlat.patch(
            text_color=Styles.WhiteBold.text_color,
            text_font=FontRoboto.ROBOTO_BOLD,
            text_size=10.8,
        ),
    )
    .add_attribute("id", "UUID", visibility="+")
    .add_attribute("status", "InvoiceState", visibility="#")
    .add_method("capture", "gw: PaymentGateway", "bool"),
    xy=(106, 29),
)

cd.connect(stripe_gw, repo, "realization", start_side="top", end_side="bottom")
cd.connect(invoice, repo, "dependency", label="<<uses>>", start_side="left",
end_side="right")

cd.draw(xy=(4.0, 4.0))
save("class_model.png")
```

10.5 Relational Schemas with ERDiagram

`ERDiagram` (`drawlib.diagrams.er`) renders Information Engineering (Crow's Foot) database schemas where each `Entity` formats its columns into three aligned sub-columns (`PK` / `FK` key badges, column name, and SQL type with `(NN)` non-null markers):

- **Table Definition (`Entity`)**: Chain `.add_column(name, type, pk=False, fk=False, nullable=True)` onto `Entity("table_name", width=38)`.
- **Crow's Foot Cardinalities & Row Anchoring**: Connect tables via `erd.connect(parent, child, cardinality="1:*", start_column="id", end_column="user_id")` (supporting `"1:1"`, `"1:0..1"`, `"0..1:1"`, `"1:*"`, `"1:1..*"`, `"0..1:*"`, and `"*:*"`). Passing `start_column` and `end_column` snaps each foreign-key line directly to the exact vertical row of the referenced column.

```
from drawlib.canvas import save, setup
from drawlib.diagrams.er import Entity, ERDiagram
from drawlib.fonts import FontRoboto, FontSourceCode
from drawlib.styles import Styles

setup(width=136, height=52)

erd = ERDiagram(
    node_style=Styles.Neutral.patch(text_font=FontSourceCode.ROBOTO_MONO,
text_size=10.0),
    header_style=Styles.PrimaryNeutral.patch(text_font=FontRoboto.ROBOTO_BOLD,
text_size=10.8),
    edge_style=Styles.DarkBold,
    edge_text_style=Styles.Dark.patch(text_size=10.0),
)

tenants = erd.add(
    Entity("tenants", width=40)
    .add_column("id", "UUID", pk=True, nullable=False)
    .add_column("slug", "VARCHAR(64)", nullable=False)
    .add_column("created_at", "TIMESTAMPTZ", nullable=False),
    xy=(30, 26),
)
api_keys = erd.add(
    Entity(
        "api_keys",
        width=42,
        header_style=Styles.PrimaryFlat.patch(
            text_color=Styles.WhiteBold.text_color,
            text_font=FontRoboto.ROBOTO_BOLD,
            text_size=10.8,
        ),
    )
    .add_column("id", "UUID", pk=True, nullable=False)
    .add_column("tenant_id", "UUID", fk=True, nullable=False)
    .add_column("key_hash", "CHAR(64)", nullable=False)
    .add_column("revoked_at", "TIMESTAMPTZ", nullable=True),
    xy=(102, 26),
)

tenants.connect(
    api_keys,
    cardinality="1:*",
    start_column="id",
    end_column="tenant_id",
    start_side="right",
    end_side="left",
)

erd.draw(xy=(4.0, 4.0))
save("er_schema.png")
```

11. Automatic Graph Layout Solvers

When diagrams grow beyond a handful of hand-placed nodes, manually maintaining `(x, y)` coordinates becomes tedious—yet opaque black-box graph renderers strip away control over iconography, typography, and design tokens. `drawlib.graph` bridges both worlds as a **pure deterministic coordinate calculator**: declare nodes, clusters, and edges topologically, call `layout = g.get_layout(...)`, and render the solved geometry using any Drawlib visual component.

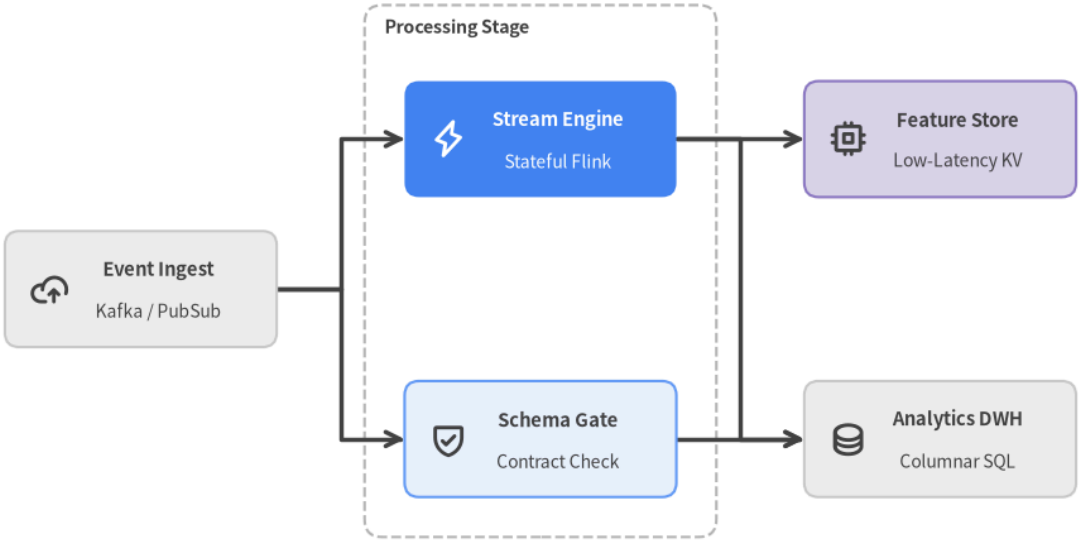


Figure 11.1: Data Lineage & Feature Pipeline DAG Solved with LayerGraph.get_layout() and Rendered onto Styled Cards

11.1 The Two-Step Layout Architecture (get_layout)

Unlike black-box layout engines, `drawlib.graph` has **no** `.draw()` **method** and stores zero visual styles. Instead, every graph follows a clean two-step separation between **geometry calculation** and **visual rendering**:

1. **Declare Topology & Solve (`layout = g.get_layout(...)`)**: Register nodes (`g.node("id", width=..., height=...)`), directed edges (`g.edge("src", "dst")`), and optional grouping containers (`g.cluster("id", nodes=[...], padding=..., header_height=...)`). Calling `g.get_layout(width=..., height=..., margin=10.0)` returns an immutable-style `GraphLayout` dictionary (`layout["id"]`).
2. **Render with Full Visual Control**: Map the solved coordinates onto `ArchitectureDiagram`, `FlowDiagram`, `SmartArts`, or primitive `shapes`, `icons`, and `lines`:
   - **Center Coordinates (`.xy` and `.local_xy`)**: Every `NodeLayout` and `ClusterLayout` exposes `.xy` (`(cx, cy)` global center) for `rectangle()`, `circle()`, `FlowDiagram` nodes, and standalone `ArchitectureDiagram` nodes, plus `.local_xy` (center relative to its parent cluster's bottom-left corner) for nodes nested inside an `ArchitectureDiagram` `NodeGroup`.
   - **Bottom-Left Corners (`.bottom_left` and `.local_bottom_left`)**: Ready to pass directly to bottom-left anchored components such as top-level `NodeGroup(..., xy=cl.bottom_left)`, nested `parent_grp.add(NodeGroup(...), xy=cl.local_bottom_left)`, `SmartArts`, and `Charts`.
   - **Routed Edge Polylines (`edge.points`, `edge.src_port`, `edge.dst_port`, `edge.mid_xy`)**: Each `EdgeLayout` in `layout.edges` provides boundary-clipped port coordinates and intermediate Manhattan waypoints ready for `lines(edge.points, arrow_head="->")`.
   - **Inspecting & Freezing Coordinates (`print(layout)` & `layout.offset()`)**: When transitioning from an automatic layout to fine-tuned manual code, call `print(layout)` (`str(layout)`) to dump a human-readable coordinate table of every solved cluster, node `.xy` / `.bottom_left`, and edge path for copy-pasting into explicit Python code, or chain `layout.offset("node_id", dx=0.0, dy=-5.0)` to nudge individual nodes while automatically recalculating attached edge ports.

11.2 Choosing Among the Five Graph Solvers

`drawlib.graph` provides five specialized layout engines inheriting from `BaseGraph`, each tailored to a distinct structural topology:

Solver Class	Core Layout Algorithm	Topology Helpers	Ideal Architectural Use Cases
`ArchitectureGraph`	2-level macro/micro container packing + 5-zone compass placement	`g.group()`, `g.cluster(..., parent=..., pos="left"|"center"|..., order=...)`	Multi-zone cloud VPCs, nested subnets, microservice ecosystems (`ArchitectureDiagram`)
`LayerGraph`	Sugiyama hierarchical DAG (cycle breaking, longest-path ranks, barycenter crossing minimization)	`g.tier(name, nodes, layer=...)`, `g.cluster()`, `direction="LR"|"TB"`	ETL/data lineage pipelines, CI/CD stages, build & package dependency DAGs
`TreeGraph`	Reingold-Tilford / Buchheim compact symmetrical subtree layout	`g.child(parent, child_id)`, `root=...`, `direction="TB"|"LR"`	Engineering org charts, ASTs, fault trees, strict single-parent taxonomies
`RadialGraph`	Concentric BFS rings + subtree-weighted angular sector allocation	`g.spoke(parent, spoke_id, ring=...)`, `hub=...`, `layout.ring_radii`	Event meshes, Pub/Sub hub-and-spoke topologies, API platform ecosystems
`GridGraph`	2D `(row, col)` matrix slot placement + corridor channel routing	`g.cell(id, row=..., col=...)`, `g.cluster_row()`, `g.cluster_column()`, `edge_routing="smart"`	Tiered service catalogs, state matrices, uniform multi-column architectures

---

11.3 Hierarchical Trees & Concentric Hub-and-Spoke Topologies

For strict hierarchies and central message brokers, `TreeGraph` and `RadialGraph` eliminate manual trigonometry and subtree width balancing:

- `TreeGraph`: Enforces acyclic single-parent tree invariants (raising a helpful `ValueError` if a diamond DAG is accidentally passed), centers every parent cleanly over its children, and routes orthogonal step connectors (`edge_routing="orthogonal"`) or straight branches (`"straight"`).
- `RadialGraph`: Places `hub` at `ring=0`, distributes connected spokes across concentric rings (`ring=1, 2, ...`), and populates `layout.ring_radii: dict[int, float]` so dashed orbital guide circles (`circle(hub.xy, radius=r, style=Styles.MutedDashed)`) align perfectly behind the spoke cards.

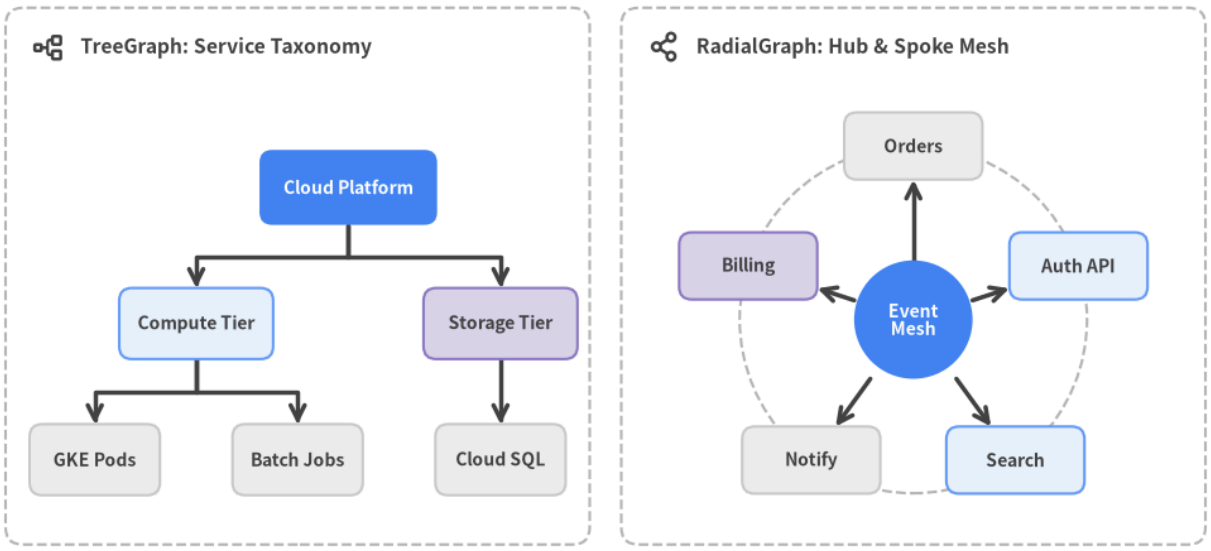


Figure 11.2: Automatic Non-Linear Layouts — Balanced Rooted TreeGraph (Left) and Concentric RadialGraph Event Mesh (Right)

11.4 Clustered Cloud Topologies & Matrix Grids

When building multi-zone cloud architectures or tabular service catalogs, `ArchitectureGraph` and `GridGraph` compute nested container boxes alongside node placements:

- `ArchitectureGraph` + `ArchitectureDiagram` **Synergy**: Pin macro zones with `pos="left" | "center" | "right" | "top" | "bottom"` and sequence nested subnets with `parent="vpc", order=1, 2`. Feed top-level clusters via `xy=layout["vpc"].bottom_left`, nested subnets via `xy=layout["app"].local_bottom_left`, and enclosed nodes via `xy=layout["api"].local_xy`—matching the exact `padding` and `(width, height)` values across both steps.
- `GridGraph` **Smart Corridor Routing**: Place nodes into `(row, col)` matrix cells with `g.cell("id", row=0, col=1)`, wrap rows or columns in dashed bands via `g.cluster_row(0, "fe_tier")` or `g.cluster_column(col, id)`, and rely on `edge_routing="smart"` to route non-adjacent connections cleanly through inter-row and inter-column corridors without crossing intermediate cards.

```python
from drawlib.canvas import save, setup
from drawlib.diagrams.architecture import ArchitectureDiagram, GcpIcon, Node, NodeGroup,
PhosphorIcon
from drawlib.graph import ArchitectureGraph
from drawlib.styles import Styles

setup(width=168, height=78)

# Step 1: Solve nested VPC & subnet coordinates with ArchitectureGraph
g = ArchitectureGraph(direction="LR", container_sep=14.0, default_node_width=24.0,
default_node_height=14.0)
g.cluster("edge", ["client"], pos="left", padding=5.0)
g.group("vpc", pos="center", padding=5.0)
g.cluster("app_tier", ["api", "worker"], parent="vpc", order=1, padding=5.0)
g.cluster("data_tier", ["db"], parent="vpc", order=2, padding=5.0)

g.edge("client", "api")
g.edge("api", "worker")
g.edge("worker", "db")

layout = g.get_layout(margin=8.0)

# Step 2: Render using .bottom_left, .local_bottom_left, and .local_xy
d = ArchitectureDiagram(
    node_style=Styles.DarkBold,
    node_text_style=Styles.DarkBold.patch(text_size=10.5),
    edge_style=Styles.DarkBold,
    edge_text_style=Styles.Dark.patch(text_size=10.0),
    node_card_style=Styles.Neutral,
)

edge_grp = d.add(NodeGroup(title="External", padding=5.0, style=Styles.MutedDashed),
xy=layout["edge"].bottom_left)
vpc_grp = d.add(NodeGroup(title="Production VPC", padding=5.0,
style=Styles.MutedDashed), xy=layout["vpc"].bottom_left)
app_grp = vpc_grp.add(NodeGroup(title="Compute", padding=5.0,
style=Styles.PrimaryNeutral), xy=layout["app_tier"].local_bottom_left)
data_grp = vpc_grp.add(NodeGroup(title="Storage", padding=5.0,
style=Styles.SecondaryNeutral), xy=layout["data_tier"].local_bottom_left)

client = edge_grp.add(Node("Client", width=24, height=14, icon=PhosphorIcon.DEVICES),
xy=layout["client"].local_xy)
api = app_grp.add(
    Node(
        "API Gateway",
        width=24,
        height=14,
        icon=GcpIcon.CLOUD_API_GATEWAY,
        style=Styles.WhiteBold,
        text_style=Styles.WhiteBold.patch(text_size=10.5),
        card_style=Styles.PrimaryFlat,
    ),
    xy=layout["api"].local_xy,
)
worker = app_grp.add(Node("Worker", width=24, height=14, icon=GcpIcon.CLOUD_RUN),
```

```
xy=layout["worker"].local_xy)
db = data_grp.add(Node("Cloud SQL", width=24, height=14, icon=GcpIcon.CLOUD_SQL),
xy=layout["db"].local_xy)

d.connect(client, api, label="HTTPS")
d.connect(api, worker)
d.connect(worker, db)

d.draw()
save("arch_graph_vpc.png")
```

12. Multi-Frame Animations & 16:9 Slide Decks

Static diagrams capture architecture snapshots, while time-stepped animations and widescreen presentations explain how systems evolve in motion. Drawlib natively compiles multi-frame **APNG** (`.png`), **Animated WebP** (`.webp`), and **dither-free Animated GIF** (`.gif`) assets via `drawlib.anim`, and transforms Markdown sources into interactive **16:9 HTML decks**, **vector PDFs**, and **editable PowerPoint** (`.pptx`) presentations via `drawlib.slide`.

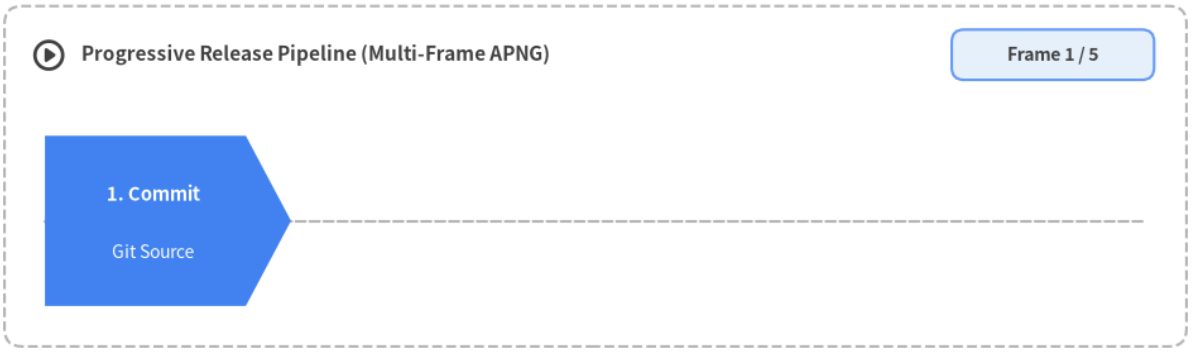


Figure 12.1: Looping Multi-Frame APNG Pipeline Walkthrough Built with Animation and ChevronProcess

12.1 Multi-Frame Animations with drawlib.anim

Canvas Lifecycle & The `Animation` API

Import `Animation` from `drawlib.anim`, call `setup(width=..., height=...)` **once before** the frame loop, and wrap each frame's drawing calls inside `with anim.frame(duration=..., clear=True):`:

- **Exact Constructor Signature**: `anim = Animation(fps=10.0, loop=0)` (or `frame_rate=10.0` as an alias for `fps`; `loop=0` plays in a continuous infinite loop).
- **No `file=` Argument on `Animation`**: `Animation` registers automatically with the active canvas and **never** accepts a `file=...` parameter. In standalone scripts, pass the destination path to `save("out.png")`, `save("out.webp")`, or `save("out.gif")`; inside Markdown ```` ```drawlib ```` blocks, set `file=name.png` (or `.webp` / `.gif`) on the fence and end the code block with `save()`.
- **Per-Frame Redraw (`clear=True`) & Final Hold**: By default, `with anim.frame(duration=None, clear=True):` clears the canvas cleanly before each frame so elements can move or change highlight styles without ghosting (`clear=False` is available for purely additive primitive sketches). Give the final frame an explicit hold (`duration=2.0` – `3.0`) so readers can absorb the completed diagram before playback loops.

Output Format	Extension	Encoding & Visual Characteristics	Best Target Platforms
APNG	`.png` / `.apng`	Lossless 24-bit RGB + 8-bit alpha; renders natively in browsers and gracefully falls back to a crisp static frame in PDFs	HTML docs (`site` / `doc`), interactive slides, printable whitepapers
Animated WebP	`.webp`	Lossless 24-bit RGB + 8-bit alpha at ~50% smaller file size (`minimize_size=True`)	High-traffic web documentation portals, mobile web apps
Animated GIF	`.gif`	`v0.3.10` dither-free 256-color global Median Cut quantization (`Image.Dither.NONE`)—zero grain noise on flat `Neutral` cards	GitHub READMEs, Slack, Notion, Confluence, X/Twitter, legacy viewers

Progressive Component Idioms (`show=False`, `draw_ratio` & Interpolation)

Every high-level Drawlib module is engineered for jitter-free frame progression:

- **Slot-Preserving Reveals (`show=False`)**: Passing `show=False` to `SmartArts` items (`ChevronProcess`, `Table`, `GridLayout`, `Cycle`) or toggling `node.show = False` on `Diagrams` (`FlowDiagram`, `ArchitectureDiagram`, `SequenceDiagram`) hides the element and its connected edges while keeping all column widths, coordinates, and spacing 100% locked across frames.
- **Smooth Chart Growth (`draw_ratio`)**: Every chart series/slice/task (`LineChart`, `BarChart`, `AreaChart`, `PieChart`, `GanttChart`) exposes `s.draw_ratio` (`0.0` to `1.0`)

and `s.draw_direction` (`"bottom_to_top"` or `"left_to_right"`) with locked `min_value` / `max_value` axes.

- **Motion & Color Interpolation**: Compute smooth trajectories (including eased ratios such as $t^2(3 - 2t)$) or waypoint steps with `drawlib.math.get_intermediate_points()` / `get_intermediate_path_points()`, and fade card fills via `drawlib.styles.get_intermediate_colors(c1, c2, num=..., include_ends=True)`.

```python
from drawlib.anim import Animation
from drawlib.canvas import save, setup
from drawlib.charts.bar import BarChart
from drawlib.styles import Styles

setup(width=110, height=64)
anim = Animation(fps=10.0, loop=0)

chart = BarChart(
    categories=["Q1", "Q2", "Q3", "Q4"],
    axis_line_style=Styles.MutedDashed,
    axis_text_style=Styles.Dark.patch(text_size=10.0),
    grid_style=Styles.MutedThin,
    width=86,
    height=44,
    title="Quarterly API Throughput (kRPS)",
    title_style=Styles.DarkBold.patch(text_size=12.5),
)
chart.configure_y_axis(min_value=0, max_value=120)
series = chart.add_series("Throughput", [42, 68, 94, 112], style=Styles.PrimaryFlat)

steps = 6
for i in range(1, steps + 1):
    t = i / steps
    eased = t * t * (3.0 - 2.0 * t)  # Smoothstep easing (0 -> 1)
    series.draw_ratio = eased
    with anim.frame(duration=2.2 if i == steps else 0.12):
        chart.draw(xy=(12, 8))

save("throughput_growth.gif")
```

12.2 16:9 Slide Decks & PowerPoint Export (drawlib.slide)

Scaffolding a presentation project with `uv run drawlib init project slide pitch -l en,ja -s google` turns numbered Markdown files (`pitch_src/<lang>/*.md`) into a `1920×1080` (`16:9`) multi-target presentation suite:

- **Dual-Layer Stage & Speaker Notes (`::: block` & `::: note`)**:
  - `::: block xy=(x, y) width=W height=H`: Places visual-first cards, 2-column splits, `---` horizontal rules, and embedded ```` ```drawlib ```` diagrams at exact top-left pixel coordinates on the `1920×1080` stage (matching diagram aspect ratios via `setup(width=W/10, height=H/10)`).
  - `::: note`: Stores full presenter scripts and technical deep dives off-stage—rendered in synchronized **Dual-Window Presenter View (`?presenter=1`, `P` / `S` key)** and exported directly into PowerPoint speaker notes while keeping the audience slide clean.
- **Runtime Slide Context & Progressive Builds (`drawlib.slide`)**:
  - `from drawlib.slide import current_slide, draw_page_number, draw_section_header` provides `current_slide.index`, `current_slide.total`, and `current_slide.text` (`"4 / 12"`), enabling dynamic footers, progressive multi-slide builds, and full-bleed chapter dividers (`variant="split" | "centered" | "banner" | "minimal"`).
  - Interactive `<canvas>` animations inside slides can be stepped live on click or `A` keypress via fence attributes (`anim_trigger=click anim_loop=once anim_pause=(1, 3)`).

````
::: block xy=(1700, 1010) width=140 height=30
```drawlib file=page.svg
from drawlib.slide import draw_page_number

draw_page_number()
```
:::

::: block xy=(80, 40) width=1760 height=60
# Zero-Trust API Gateway Rollout
:::

::: block xy=(80, 140) width=720 height=820
## Key Rollout Milestones

- **Edge mTLS**: Mutual certificate pinning
- **JWT Cache**: Sub-millisecond Redis lookup
- **Canary Gate**: Automated SLO rollback

---

*Active Deck Position: Slide 3*
:::

::: block xy=(840, 140) width=1000 height=820
```drawlib file=gateway_stage.svg
from drawlib.canvas import clear, save, setup
from drawlib.shapes import rectangle
from drawlib.slide import current_slide
from drawlib.styles import Styles

clear()
setup(width=100, height=82)
rectangle(
    (50, 41),
    width=76,
    height=34,
    style=Styles.PrimaryFlat.patch(shape_r=2.5),
    text=f"Gateway Topology ({current_slide.text})",
    text_style=Styles.WhiteBold.patch(text_size=14.5),
)
save()
```
:::

::: note
Walk through the three milestones on the left, then highlight how `current_slide` binds
slide numbering automatically during HTML, PDF, and `.pptx` compilation.
:::
````

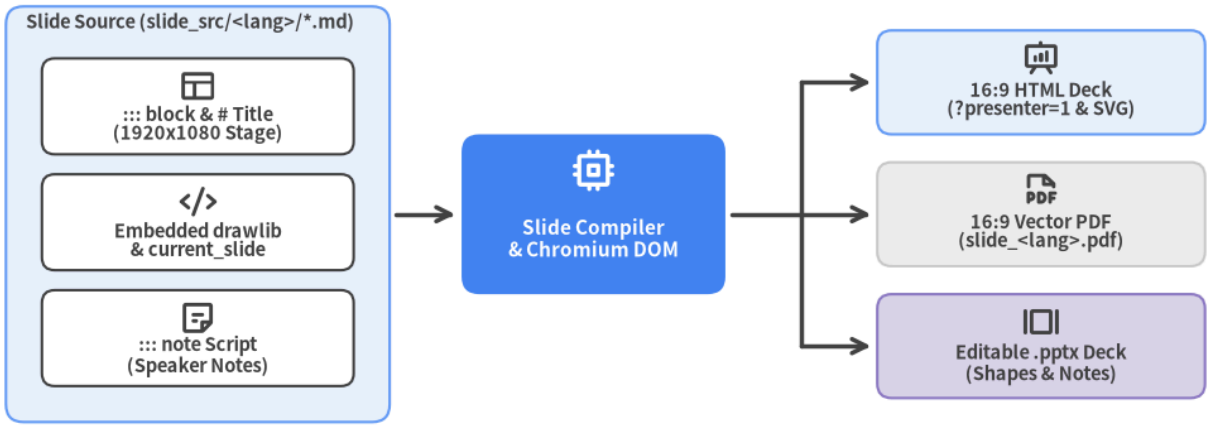


Figure 12.2: 16:9 Slide Deck Compilation Anatomy — Markdown Stage & Notes to HTML, Vector PDF, and Editable .pptx

Scaffolding & Building HTML, PDF, and Editable PowerPoint (`.pptx`)

Every scaffolded `slide` project includes ready-to-run build scripts for interactive HTML, 16:9 vector PDF, and native Microsoft PowerPoint export (`drawlib build pptx` translates `::: block` cards, text boxes, lists, tables, and `::: note` speaker notes into editable `960×540 pt` PowerPoint elements while embedding diagrams at `240 DPI`):

```
# 1. Scaffold a bilingual 16:9 slide deck project
uv run drawlib init project slide pitch -l en,ja -s google

# 2. Compile interactive 16:9 HTML deck + 1920x1080 vector PDF + standalone assets
./pitch_src/build.sh

# 3. Export native editable PowerPoint decks (pitch_en.pptx & pitch_ja.pptx)
./pitch_src/build_pptx.sh
# Equivalent CLI command:
uv run drawlib build pptx pitch_src/ -o pitch.pptx -s pitch_src/styles.py -u
pitch_src/utils.py
```

13. Documentation as Code & The 5 Project Archetypes

Writing standalone diagrams is only half the story. In real engineering organizations, architecture visuals live inside multi-chapter design documents, searchable API portals, RFC whitepapers, and executive slide decks that must stay synchronized across languages and output formats.

Drawlib elevates **Illustration as Code** into **Illustrated Documentation as Code**: a unified project compiler where pure-Python diagrams and Markdown prose live side-by-side in Git and compile deterministically into static HTML websites, A4 vector PDFs, editable Microsoft Word (`.docx`) specifications, 16:9 HTML/PDF slide decks, native PowerPoint (`.pptx`) files, and animated image suites.

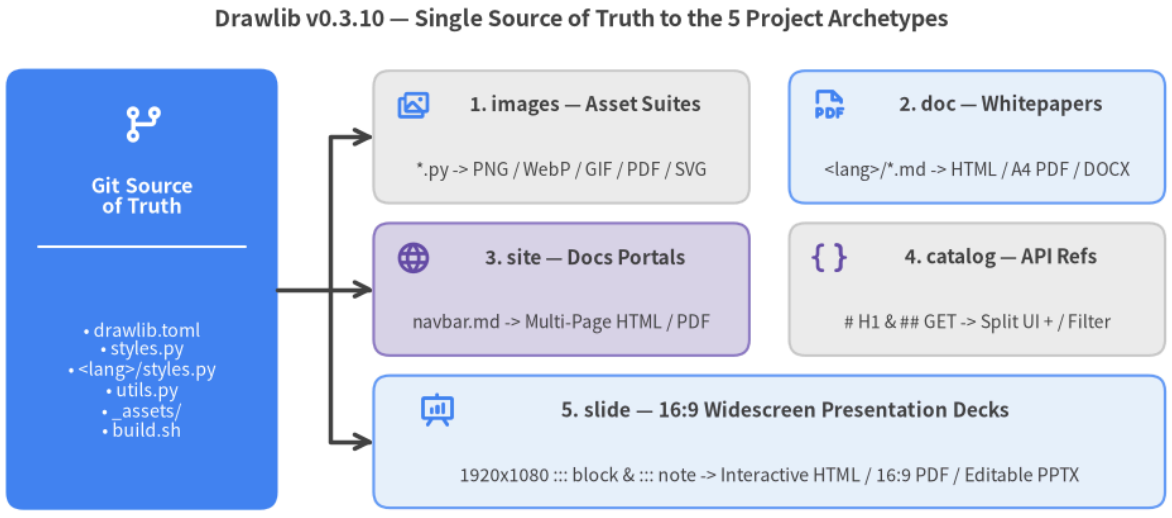


Figure 13.1: Drawlib's Single Source of Truth Compiling into the 5 Project Archetypes (v0.3.10)

13.1 The Project-First Rule (`drawlib init project`)

When starting any visual or documentation task—even a single architecture figure—always follow Drawlib's **Project-First Rule**: scaffold a structured `<target>_src/` workspace with `drawlib init project` rather than scattering ad-hoc standalone `.py` scripts or hand-crafting folders.

Scaffolding a project up front establishes five engineering guarantees out of the box:

- **Canonical Manifest (`drawlib.toml`)**: Declares the project archetype (`images`, `doc`, `site`, `catalog`, or `slide`), visual preset (`default`, `google`, `monochrome`), multi-language `[i18n]` routing, and `[seo]` metadata in one declarative file.
- **Centralized Theme & CJK Font Safety (`styles.py` & `<lang>/styles.py`)**: Passing `-l ja` or `-l en,ja` automatically configures `Styles` and regional font overrides (`FontJapanese`, `FontChinese`, `FontKorean`) so every diagram shares one palette without repeating font boilerplate.
- **DRY Component Macros (`utils.py`)**: Top-level helper functions defined in `utils.py` become importable everywhere via `from drawlib.utils import <helper>`, keeping recurring node cards and connectors consistent across dozens of chapters.
- **Portable Static Assets (`_assets/`)**: Provisions `_assets/favicon.png` and provides a root directory for brand logos and custom `.ttf` fonts that Drawlib resolves and SHA-256 hashes automatically.
- **Reproducible One-Command Builds (`build.sh` & `build_*.sh`)**: Generates executable build scripts (`build.sh`, `build_html.sh`, `build_pdf.sh`, `build_docx.sh`, `build_pptx.sh`, `serve.sh`) while strictly separating `<target>_src/` (your editable source of truth) from read-only compiled outputs.

Use `drawlib init project <type> [target] -l <langs> -s <style>` to initialize any of the five archetypes (and delete starter `sample*.py` files once you add your own diagrams):

```
# Inspect all 5 starter project templates
uv run drawlib init project list

# 1. Standalone Python diagram & animation repository (images_src/)
uv run drawlib init project images diagrams -l en -s google

# 2. Bilingual linear technical whitepaper -> HTML + A4 PDF + MS Word (.docx)
uv run drawlib init project doc whitepaper -l en,ja -s google

# 3. Multi-page documentation portal with sidebar navigation (docs_src/)
uv run drawlib init project site docs -l en,ja -s default

# 4. Searchable API / CLI / schema reference dictionary (catalog_src/)
uv run drawlib init project catalog api_ref -l en,ja -s google

# 5. Bilingual 16:9 presentation slide deck -> HTML + PDF + PowerPoint (.pptx)
uv run drawlib init project slide pitch -l en,ja -s google
```

13.2 Embedded Markdown Diagrams & 720pt Typography Parity

Inside `doc`, `site`, `catalog`, and `slide` Markdown files (`<target>_src/<lang>/**/*.md`), Python illustrations are embedded directly inline using fenced `drawlib` blocks. During compilation, Drawlib executes each block in an isolated canvas state, caches the rendered output in `.drawlib/cache.db`, and replaces the fence with responsive figure markup.

```drawlib align=center code=fold file=gateway_flow.png caption="Figure 13.3: Regional API Gateway Topology"
from drawlib.canvas import save, setup
from drawlib.lines import line
from drawlib.shapes import rectangle
from drawlib.styles import Styles

setup(width=136, height=44)

rectangle((28, 22), width=32, height=18, style=Styles.Neutral, text="Client App")
rectangle(
    (68, 22),
    width=32,
    height=18,
    style=Styles.PrimaryFlat,
    text="API Gateway",
    text_style=Styles.WhiteBold,
)
rectangle((108, 22), width=32, height=18, style=Styles.SecondaryNeutral, text="Auth Service")

line((44, 22), (52, 22), arrow_head="->", style=Styles.DarkBold)
line((84, 22), (92, 22), arrow_head="->", style=Styles.DarkBold)
save()
```

Every embedded `drawlib` fence supports four header attributes that govern file naming, layout, and code disclosure:

Fence Attribute	Supported Values	Recommended Usage
`file=`	`<stem>.png` / `.webp` / `.svg`	**Mandatory.** Assigns a deterministic filename (`file=arch.png` for shared diagrams, or `file=arch_ja.png` for localized CJK diagrams) so `drawlib show <md> <file>` can target it by name.
`caption=`	`"Figure X.Y: ..."`	**Mandatory on docs.** Renders an accessible `<figcaption>` in HTML/PDF and figure caption paragraph in `.docx`.
`align=`	`center` | `left` | `right`	Controls horizontal figure alignment inside the document column (`align=center` standard).
`code=`	`hide` | `fold` | `show`	`code=hide` (default) emits only the figure for clean whitepapers; `code=fold` renders the figure first with a collapsible `<details>` code drawer (**mandatory for Top Hero diagrams at Lines 5–15**); `code=show` displays syntax-highlighted Python above the figure for API tutorials.

Why Omitting Pixel `width=` Caps Preserves `720pt` Typography

Drawlib's rendering engine normalizes every canvas (`setup(width=W, height=H)`) to a physical width of `10 inches = 720 points` regardless of whether `W=100` or `W=160` . When that image is displayed inside an HTML column of width W_{HTML} (typically `~880px–960px`), an in-diagram `text_size` (in points) appears on screen at:

$$\text{Rendered CSS Font Size (px)} = \text{text_size} \times \frac{W_{\text{HTML}}}{720}$$

- **Without pixel caps (`100%` column width, `~900px`):** `text_size=11.0` renders at `11.0 × 900 / 720 = 13.75px` , harmonizing naturally with `16px` HTML body prose and A4 PDF text.
- **With hardcoded narrow caps (`width=500px` or `600px` — Forbidden):** Shrinking the image container to `500px` crushes `text_size=11.0` down to `7.6px` , making labels unreadable even when the standalone PNG looked fine in isolation. Always omit pixel `width=` caps on fences and keep `text_size >= 10.0` .

13.3 Comparing the 5 Project Archetypes

Drawlib `v0.3.10` provides five purpose-built project archetypes. All four Markdown-driven archetypes (`doc` , `site` , `catalog` , `slide`) use the exact same `<target>_src/<lang>/` locale directory convention so teams can add new languages at any time without moving existing files.

Archetype	Source Layout	Navigation & Structure	Compiled Deliverables	Primary Engineering Use Cases
`images`	`images_src/*.py`	Flat or nested Python scripts; AST duplicate output check	`.png` (incl. `APNG`), `.webp` , `.gif` , `.pdf` , `.svg`	Architecture asset libraries, README figures, CI-generated charts, multi-frame protocol animations
`doc`	`doc_src/<lang>/*.md` (`00_cover.md` , `01_*` ...)	Lexicographical chapter merge + `--generate-index` TOC (`no navbar.md`)	Single-page `HTML` book, A4 vector `PDF` , editable Word `.docx` , GitHub `MD`	Engineering RFCs, architecture whitepapers, onboarding manuals, printable specs & stakeholder redlining

Archetype	Source Layout	Navigation & Structure	Compiled Deliverables	Primary Engineering Use Cases
`site`	`docs_src/<lang>/**/*.md` + `<lang>/navbar.md`	Persistent multi-section sidebar (`navbar.md`), dark mode, search	Multi-page `HTML` portal, per-page `PDF` , GitHub `MD` , `llms.txt`	Product documentation portals, developer guides, multi-team architecture wikis
`catalog`	`catalog_src/<lang>/*.md` *(auto-indexed)*	Sticky 2-column split/outline UI from `# H1` & `## H2` + instant `/` live filter	Interactive 2-column `HTML` dictionary, GitHub `MD` , `llms.txt`	REST/gRPC API references, CLI flag manuals, event/metric catalogs, database schema dictionaries
`slide`	`slide_src/<lang>/*.md` (`01_title.md` ...)	`1920×1080` `::: block` stage + `::: note` speaker script + Presenter View	`16:9` interactive `HTML` deck, `16:9` vector `PDF` , editable PowerPoint `.pptx`	Technical keynotes, architecture review decks, sprint demos, customer executive briefings

Key capabilities distinguishing each archetype in daily workflows:

1. `images` **(Standalone Diagrams & Animations)**: Each `*.py` script calling `save()` automatically emits `<script_stem>.png` (or `.webp` / `.gif` for `Animation()` loops). Before running any script, `drawlib build image` parses all files via Python's `ast` module and aborts immediately if two scripts would collide on the same output filename. When writing multiple files inside one script, call `clear()` before each subsequent `setup()` .
2. `doc` **(Linear Multi-Chapter Books, PDF & Word** `.docx` **)**: Merges numbered chapters (`00_cover.md` , `01_overview.md` , ...) in sorted order, injecting CSS/Word page breaks (`--page-break`) and a localized Table of Contents (`--generate-index`) right after `00_cover.md` . Alongside `./doc_src/build.sh` (`HTML` + A4 `PDF`), run `./doc_src/build_docx.sh` (`drawlib build docx`) whenever stakeholders need a native editable `.docx` file.
3. `site` **(Multi-Page Documentation Portals)**: Driven by `<lang>/navbar.md` , compiling nested Markdown trees into responsive static portals with light/dark theme switching, localized sidebars, syntax highlighting, and cross-page link rewriting (`.md` -> `.html`).
4. `catalog` **(Searchable Reference Dictionaries)**: Eliminates manual sidebar maintenance (`1 Markdown = 1 HTML Page`). Each file's `# H1` becomes a collapsible sidebar category, while every `## H2` becomes a scroll-spied `#slug` entry supporting instant `/` keyboard live filtering and color-coded semantic prefix badges—including HTTP verbs (`## `GET` /v1/users/<id>` , `## `POST` /v1/orders`), code symbols (`## `def` setup()` , `## `class` BarChart`), and infrastructure entities (`## `service` auth-api` , `## `table` accounts` , `## `cmd` drawlib build`).

5. `slide` (`16:9` **Widescreen Decks & Editable** `.pptx`): Separates concise visual slides (`::: block xy=(x, y) width=W height=H` on a `1920×1080` canvas) from rich presenter scripts (`::: note`). In addition to dual-screen HTML Presenter View and `16:9` vector PDFs, `./slide_src/build_pptx.sh` (`drawlib build pptx`) measures the live Chromium DOM to emit native editable PowerPoint shapes, text boxes, tables, speaker notes, and `240 DPI` diagrams.

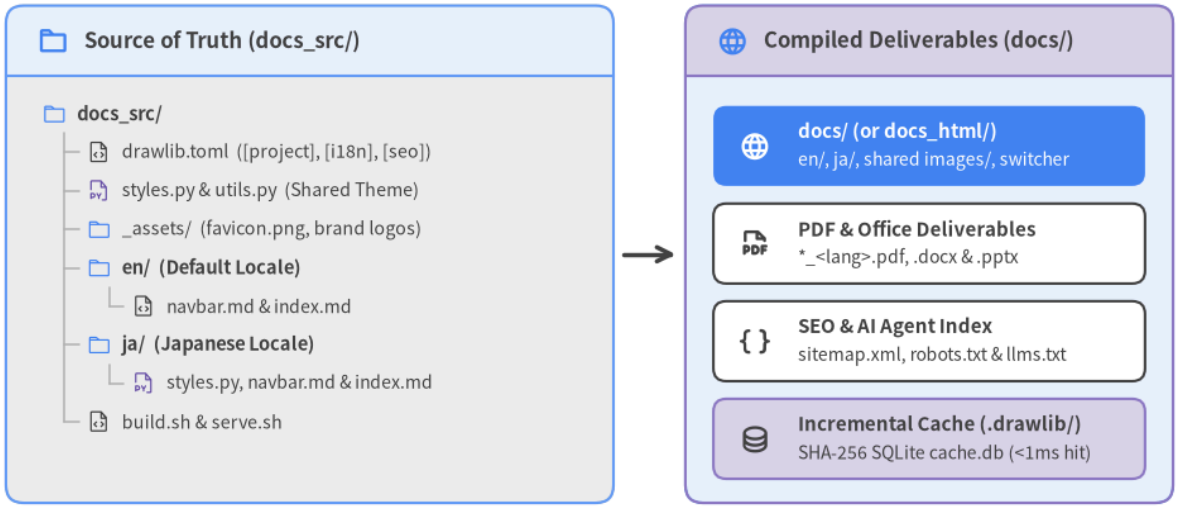


Figure 13.2: Multi-Locale Project Directory Anatomy (TreeNode Source Layout Compiling into docs/ Deliverables)

13.4 Configuring `drawlib.toml`: `[project]`, `[i18n]`, and `[seo]`

Every Markdown-based Drawlib project (`doc`, `site`, `catalog`, `slide`) is governed by `drawlib.toml` at the root of `<target>_src/`. Centralizing metadata in `drawlib.toml` eliminates fragile CLI flag duplication across shell scripts and CI workflows:

```toml
[project]
name = "platform_docs"
type = "site"                 # "images" | "doc" | "site" | "catalog" | "slide"
title = "Cloud Platform Architecture"
description = "Unified architecture portal and API guide."
style = "google"              # "default" | "google" | "monochrome"

[i18n]
default_lang = "en"
langs = ["en", "ja"]          # Always present on doc/site/catalog/slide (even ["en"])

[i18n.labels]
en = "English"
ja = "日本語"

[i18n.descriptions]
en = "Cloud Platform architecture documentation and operational runbooks."
ja = "クラウドプラットフォームのアーキテクチャ設計書および運用ガイド。"

[seo]
site_url = "https://docs.example.com"
og_image = "_assets/ogp.png"
twitter_card = "summary_large_image"
sitemap = true
robots = true
llms_txt = true
```

Multi-Language Skeleton Parity (`[i18n]`) & AI Discoverability (`[seo]`)

When `langs` contains two or more locales (such as `["en", "ja"]`), Drawlib runs **four mechanical pre-flight parity checks** before rendering any page, catching translation drift at build time rather than in production:

- **1. File Set & `navbar.md` Parity**: Every locale folder (`en/` , `ja/`) must contain the exact same relative `.md` file paths, and (for `site` projects) `navbar.md` must have identical section counts and normalized link targets.
- **2. Structural Skeleton Parity**: Each translated page must match `default_lang` in its ordered sequence of `#` / `##` / `###` headings (`H1` – `H3`), prose/table/list slots (`MarkdownText`), non-drawlib fenced code block languages (`CodeBlock`), and `drawlib` block attributes (`align` , `code` , `caption` presence). Consecutive paragraphs, lists, and tables collapse into a single `MarkdownText` slot so translators can write natural prose freely.
- **3. Shared vs. Localized Diagram Routing**: If `ja/` reuses the same filename (`file=arch.png`), Drawlib verifies that the Python code is byte-for-byte identical and

renders the image **once** across all locales. When diagram labels are translated into Japanese, pair a locale-suffixed filename (`file=arch_ja.png`) with `ja/styles.py` (`FontJapanese`).

- **4. Automated SEO & `llms.txt` Generation (`[seo]`)**: Setting `[seo]` injects `<link rel="canonical">` , `hreflang` cross-locale tags, Open Graph (`og:*`), and Twitter Card metadata into every HTML page, while emitting `sitemap.xml` , `robots.txt` , and an `llmstxt.org` -compliant `llms.txt` index at the HTML root so AI coding assistants and search crawlers can index your documentation structure immediately.

14. CLI Toolchain & Autonomous AI Agent Workflows

Drawlib is engineered from the ground up for both human developers and multimodal AI coding assistants. Because every diagram, document, and slide deck compiles headlessly through a deterministic Command Line Interface (`drawlib`), AI agents never have to guess pixel coordinates blindly or rely on manual GUI clicks.

Instead, an AI coding agent reads on-demand API manuals (`drawlib rules show`), scaffolds a project (`drawlib init`), renders single-diagram coordinate grids (`drawlib show -g`), visually inspects its own output multimodally, self-corrects overlaps in a single pass, and validates every link before delivery (`drawlib serve --check`).

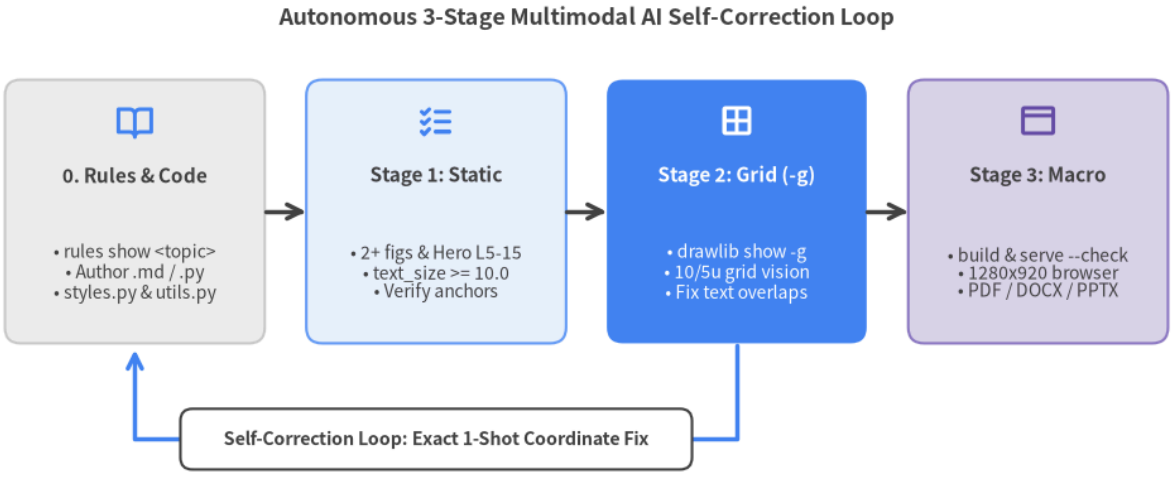


Figure 14.1: The 3-Stage Autonomous AI Multimodal Self-Correction Loop

14.1 Bootstrapping Projects & AI Agent Rules (`drawlib init`)

The `drawlib init` command group handles two foundational setup tasks: scaffolding project workspaces (`drawlib init project`) and provisioning AI coding assistant instruction files (`drawlib init agent`).

Running `drawlib init agent` installs Drawlib's architectural conventions, `720pt` typography math, coordinate anchor rules, and 3-stage verification protocol directly into your AI assistant's native rule directory. Drawlib `v0.3.10` provides 9 built-in agent targets—and because `AGENTS.md` is an open multi-tool standard, it also works seamlessly with `AGENTS.md`-compatible tools (including OpenCode and Kiro):

CLI Target	AI Coding Assistant	Generated Workspace Path	Rule Format & Loading Behavior
`antigravity`	Google Antigravity	`.agents/rules/drawlib.md`	YAML frontmatter (`trigger: model_decision` or `always_on`)
`gemini`	Gemini CLI	`GEMINI.md`	Non-destructive HTML comment marker merge (`~/.gemini/GEMINI.md` with `--global`)
`claude`	Claude Code	`.claude/rules/drawlib.md`	Modular Markdown rule coexisting cleanly with root `CLAUDE.md`
`cursor`	Cursor	`.cursor/rules/drawlib.mdc`	MDC frontmatter (`description`, `alwaysApply`)
`codex`	OpenAI Codex / `AGENTS.md`	`AGENTS.md`	Non-destructive HTML marker merge; shared by Codex, Kiro & `AGENTS.md` agents
`copilot`	GitHub Copilot	`.github/instructions/drawlib.instructions.md`	YAML frontmatter (`applyTo: "**/*_src/**, **/*.md, **/*.py"`)
`windsurf`	Windsurf	`.windsurf/rules/drawlib.md`	Cascade workspace rule with YAML trigger frontmatter
`cline`	Cline	`.clinerules/drawlib.md`	Individually toggleable rule inside `.clinerules/`
`roo`	Roo Code	`.roo/rules/drawlib.md`	Modular workspace or `--global` Roo rule file

Pass `--detect` to inspect existing configuration folders in any repository and generate matching rule files automatically, or `-m auto` (default) so the rule activates on demand without consuming context tokens on unrelated backend tasks:

```
# List all supported AI coding agent targets and destination paths
uv run drawlib init agent list

# Auto-detect configured AI assistants in the current repo and install rules
uv run drawlib init agent --detect

# Explicitly generate modular rules for specific assistants (or 'all' for all 7 modular files)
uv run drawlib init agent antigravity claude cursor copilot

# Force always-on loading for dedicated documentation repositories
uv run drawlib init agent antigravity --mode always

# Install user-wide global Drawlib rules into ~/.claude/rules/ or ~/.gemini/config/rules/
uv run drawlib init agent claude --global
```

14.2 Compilation, Incremental Caching & Dev Server

Once a project is scaffolded, five core CLI command groups drive day-to-day authoring, rapid visual inspection, multi-target publishing, and CI quality gates:

Command Group	Key Subcommands & Flags	Core Purpose
`drawlib build`	`html`, `pdf`, `docx`, `slide`, `pptx`, `markdown`, `image` (`-s`, `-u`, `--no-cache`)	Compiles `<target>_src/` into HTML portals/books/decks, vector PDFs, editable `.docx` / `.pptx` Office files, GitHub Markdown, and batch PNG/WebP/GIF images.
`drawlib show`	`<file> [target] -s styles.py -u utils.py -g -o preview.png`	Renders a single `.py` script or named Markdown block (`file=arch.png`) headlessly to disk with an optional `10/5`-unit coordinate grid (`-g`).
`drawlib serve`	`<html_dir> [-p 8000] [--no-browser] [--check]`	Launches a zero-dependency local HTTP preview server, or runs `--check` in CI to verify zero broken `<a href>` links or missing `<img src>` assets.
`drawlib cache`	`list`, `clear --images`, `clear --all`, `download --all`, `download --browser`	Manages `.drawlib/cache.db` SQLite diagram hashes and pre-downloads fonts, 34 icon suites, GeoJSON maps, and Playwright headless Chromium.

Command Group	Key Subcommands & Flags	Core Purpose
`drawlib rules`	`list`, `show <topic> [--raw | --rebuild]`, `build --all`, `clear`	Built-in self-documenting manual (`overview`, `cli`, `style-guide`, `review-guide`, `lib-*`) with on-demand reference illustrations for humans and LLMs.

Inside `.drawlib/cache.db`: SHA-256 Content-Addressed Caching

Re-executing 50 Python diagrams on every prose typo fix would slow down iterative writing. Before running any `drawlib` block or `.py` script, Drawlib computes a deterministic **SHA-256 digest** across four inputs:

1. The exact Python source code of the diagram block or script.
2. The full contents of `styles.py` (`-s`) and `utils.py` (`-u`).
3. Local static files referenced as string literals (such as `"_assets/logo.png"`).
4. Target output format (`png`, `webp`, `svg`), relative path, and slide context.

On a cache hit in `.drawlib/cache.db`, Drawlib restores the compiled binary in **under 1 millisecond** without invoking Matplotlib. The database automatically writes `.drawlib/.gitignore` (`*`), invalidates entries when `drawlib` or `matplotlib` versions upgrade, and evicts the oldest 50% of blobs above `1 GiB`. If your diagram imports an external custom Python module outside `styles.py` and `utils.py`, pass `--no-cache` or run `drawlib cache clear --images`:

```
# 1. Inspect a single named Markdown diagram headlessly with the 10/5-unit grid (-g)
uv run drawlib show doc_src/en/01_overview.md system_arch.png \
    -s doc_src/styles.py -u doc_src/utils.py -g --no-cache -o
.drawlib/scratch/arch_grid.png

# 2. Multi-target builds (HTML, A4 PDF with TOC, MS Word .docx, 16:9 Slides & PowerPoint
.pptx)
uv run drawlib build html docs_src/ -o docs_html/ -s docs_src/styles.py -u
docs_src/utils.py
uv run drawlib build pdf doc_src/ -o manual.pdf --page-break --generate-index -s
doc_src/styles.py
uv run drawlib build docx doc_src/ -o manual.docx --page-break --generate-index --dpi
200
uv run drawlib build slide slide_src/ -o slide_html/ -s slide_src/styles.py
uv run drawlib build pptx slide_src/ -o slide.pptx --dpi 240 -s slide_src/styles.py
uv run drawlib build markdown docs_src/ -o docs_markdown/
uv run drawlib build image images_src/ -f webp

# 3. CI pre-warming, SQLite image cache reset, broken-link check & on-demand rules
lookup
uv run drawlib cache download --all
uv run drawlib cache clear --images
uv run drawlib serve docs_html/ --check
uv run drawlib rules list
uv run drawlib rules show lib-smartarts-overview --raw
```

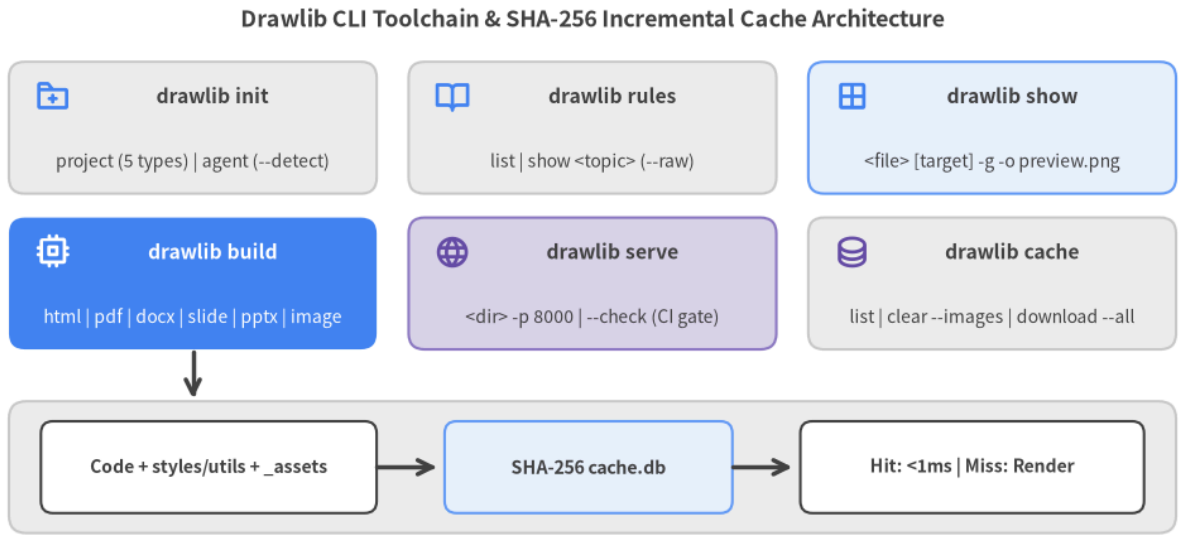


Figure 14.2: Drawlib CLI Command Ecosystem & SHA-256 SQLite Build Cache Pipeline

14.3 Deep-Dive: The 3-Stage Autonomous AI Review Loop

Inspecting a single PNG in isolation is not enough to guarantee publication quality: a diagram might have subtle label overlaps inside the canvas (micro-geometry defects) or shrink to microscopic text inside an HTML column or 16:9 slide (macro-page defects). AI agents eliminate both failure modes by executing all three verification stages before finishing:

1. **Stage 1 — Static Code & Anchor Pre-Check**:
   - Confirm `>= 2` embedded `drawlib` diagrams per documentation page and a **Top Hero diagram starting on Lines 5–15** with `code=fold` (so readers see a visual summary above the fold).
   - Confirm zero pixel width caps (`width=500px` / `600px` omitted), `text_size >= 10.0`, strictly relative Markdown paths, and exact coordinate anchor semantics (`Center` vs. `Bottom-Left` vs. `Top-Left`).
2. **Stage 2 — Micro-Geometry Grid Review (`drawlib show -g`)**:
   - Export each created or modified diagram headlessly with `-g` (`uv run drawlib show <file> <target.png> -s styles.py -u utils.py -g -o .drawlib/scratch/<unique>_grid.png`) and inspect the PNG with multimodal vision.
   - Because `-g` overlays **10-unit major grid lines** (labeled numerically) and **5-unit minor dashed lines**, the agent reads off exact coordinate adjustments in a single iteration—fixing any text border clipping, arrowhead collisions, perimeter margins (`< 4–6` units), or rainbow color overuse (`< 50%` neutral fills).
3. **Stage 3 — Macro-Page & Deliverable Verification**:
   - Compile the project (`./<target>_src/build.sh`), run `uv run drawlib serve <target>_html/ --check` to verify zero broken internal links or missing image assets, and capture a `1280×920` **headless Chromium screenshot** of the rendered HTML (or inspect compiled `PDF`, `DOCX`, and `PPTX` pages).
   - Confirm that the Top Hero sits cleanly above the fold and that in-diagram typography matches surrounding `16px` HTML prose (or `26px` slide prose on `1920×1080` decks), then clean up `.drawlib/scratch/`.

14.4 Scaling with Parallel Multi-Agent Delegation

When creating or updating multi-chapter whitepapers (`doc`), multi-section portals (`site` / `catalog`), or 20-slide decks (`slide`), loading dozens of high-resolution grid preview PNGs into a single agent's context window wastes tokens and latency. Drawlib's modular file architecture maps cleanly onto a **3-Phase Orchestrator–Subagent Workflow**:

- **Phase 1 — Parent Orchestrator Scaffolds Shared Foundation**: The parent agent runs `drawlib init project` , finalizes shared `drawlib.toml` , `styles.py` (`ja/styles.py`), `utils.py` , `_assets/` , and `<lang>/navbar.md` or numbered chapter filenames first.
- **Phase 2 — Parallel Subagents Author Disjoint Files (`--no-cache`)**: Each subagent receives exclusive ownership of disjoint `.md` or `.py` files (treating shared config files as read-only) and runs Stage 1 + Stage 2 grid reviews (`drawlib show ... -g --no-cache -o .drawlib/scratch/<worker>_<stem>.png`) using **unique scratch filenames**—never running `./build.sh` concurrently so `.drawlib/cache.db` never experiences SQLite lock contention.
- **Phase 3 — Parent Runs Unified Build & Link Gate**: Once all subagents complete Stage 2 visual verification, the parent orchestrator runs `./<target>_src/build.sh` once, validates cross-page links with `drawlib serve --check` , performs Stage 3 browser/PDF review, and removes `.drawlib/scratch/` .

15. Visual Design Rules & Production Checklist

Great technical diagrams communicate system architecture in under three seconds—not by adding decorative colors, but through disciplined typography scaling, calm neutral contrast, predictable coordinate anchors, and reusable component macros.

This final chapter consolidates Drawlib's engineering design laws into a single-page production cheat sheet that human authors and AI coding agents can consult before shipping any diagram, whitepaper, portal, API catalog, or 16:9 presentation deck.

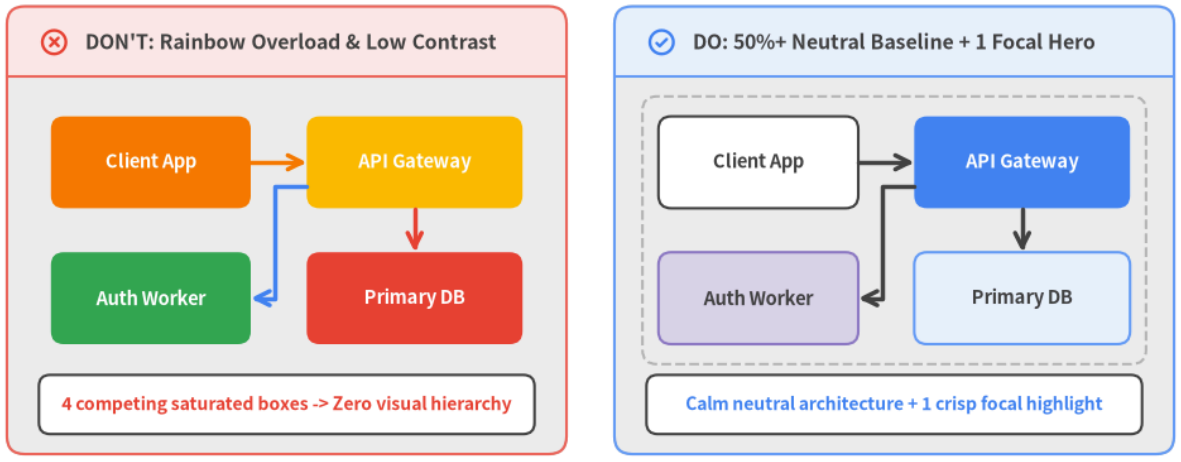


Figure 15.1: Color & Contrast Discipline — DON'T (Rainbow Overload) vs. DO (50%+ Neutral Baseline + 1 Focal Highlight)

15.1 The `720pt` Typography Scale & Width Discipline

Because Drawlib maps every canvas width (`setup(width=W, height=H)`) to `10 inches = 720 points` , rendered font size in HTML or PDF depends strictly on `text_size × ContainerWidth / 720` . Hardcoding narrow pixel caps like `width=500px` or `width=600px` on a Markdown

fence shrinks the container below `720pt`, breaking visual parity with `16px` HTML prose. Always omit pixel `width=` caps on documentation fences and follow this typography scale:

Hierarchy Role	Recommended `text_size`	Recommended `Styles` Token	Rendered Size in `~900px` Column
Canvas Title	`14.0 – 16.0`	`Styles.DarkBold.patch(text_size=14.5)`	`~17.5px – 20.0px` (Clear banner lead)
Section / Group Header	`12.0 – 13.5`	`Styles.DarkBold.patch(text_size=12.5)`	`~15.0px – 16.9px` (Matches subheadings)
Node / Card Body	`10.5 – 12.0`	`Styles.DarkBold` (light) / `Styles.WhiteBold` (`*Flat`)	`~13.1px – 15.0px` (Matches `16px` prose)
Edge / Axis Labels	`10.0`	`Styles.Dark` or `Styles.Muted`	`~12.5px` (Clean protocol/unit caption)
Strict Floor (Ticks / Code)	`9.5` **(Never** `< 9.5`**)**	`Styles.Dark` / `SourceCode`	`~11.9px` (Minimum legible threshold)

> **16:9 Slide Deck Adjustment (`1920×1080` Stage)**: Slide prose defaults to `26px`—much larger than `16px` web docs. When placing a diagram inside a half-width 2-column `:::block` (`width=960..1040`), raise node/axis `text_size` to `13.5 – 15.5` (and titles to `16.0 – 18.5`) while widening boxes by `25%–40%` so labels match adjacent slide bullets.

15.2 The `50%+` Neutral Baseline & Contrast Discipline

Human-crafted architecture diagrams use color sparingly to direct attention rather than decorate every box:

- **Ground `>= 50%` of Shapes in Calm Neutrals**: Build the majority of service cards, worker nodes, and containers with `Styles.Neutral`, `Styles.PrimaryNeutral`, `Styles.SecondaryNeutral`, or `Styles.White` (and `Styles.MutedDashed` for VPC/cluster boundaries).
- **Reserve Saturated `*Flat` Fills for `1–2` Focal Nodes**: Use `Styles.PrimaryFlat` or `Styles.AccentFlat` exclusively on the core system under discussion so the reader's eye locks onto the focal component immediately.
- **Strict Luminance Contrast Pairing**:

    - On saturated solid fills (`Styles.PrimaryFlat`, `Styles.AccentFlat`, `Styles.DangerFlat`), **always** pair `Styles.WhiteBold` (titles) and `Styles.White` (subtitles/icons).
    - On neutral or light fills (`Styles.Neutral`, `Styles.PrimaryNeutral`, `Styles.SecondaryNeutral`, `Styles.White`), **always** pair `Styles.DarkBold` and `Styles.Dark` (avoid chromatic blue/purple body text on white cards).
- **Neutral Connectors by Default**: Use `Styles.DarkBold` (or `Styles.Dark`) for standard sequential and structural arrows, reserving `Styles.DangerBold` (errors/rollbacks), `Styles.SuccessBold` (verified commits), and `Styles.PrimaryBold` (highlighted critical path) for intentional semantic contrast.
- **PascalCase Token Imports**: Always write `from drawlib.styles import Colors, Styles` —never lowercase `styles` or `colors`, which shadows module namespaces and bypasses `styles.py` overrides.

15.3 Master Coordinate Anchor Cheat Sheet

Every Drawlib canvas uses Cartesian coordinates with `(0, 0)` at the **bottom-left** (`+X` right, `+Y` up), but individual component families interpret their `xy` placement parameter using one of three anchor conventions. Memorizing—or verifying—this table prevents 100% of off-by-half layout shifts:

Anchor Convention	Drawlib Families & Classes	How `xy` Behaves	Horizontal / Vertical Span

Center `(cx, cy)`	`shapes.*` (`rectangle`, `circle`, `cylinder`, ...)
	`text.text()`, `icons.*` (`phosphor`, `gcp`, `aws`, `azure`, ...)
	`images.image()`
	`smartarts.Cycle`, `smartarts.MindMapNode`
	`diagrams.*` `Node(..., xy=(cx, cy))`
		Geometric **center** of the element	`x: [cx - w/2, cx + w/2]`
			`y: [cy - h/2, cy + h/2]`

Bottom-Left `(x0, y0)`	`charts.*` (all 10 quantitative charts) `diagrams.*` `d.draw(xy=(x0, y0))` & `NodeGroup(..., xy=(x0, y0))` Most `smartarts.*` (`BoxList`, `CardList`, `ChevronProcess`, `GridLayout`, `Pyramid`, `KpiCards`, `Roadmap`, `Panel`, `GeoMap`, `*Calendar`)	**Bottom-left corner** (`+x` right, `+y` up); center on canvas `W` via `x0 = (W - width) / 2`	`x: [x0, x0 + width]` `y: [y0, y0 + height]`

Anchor Convention	Drawlib Families & Classes	How `xy` Behaves	Horizontal / Vertical Span

Top-Left `(x0, y_top)`	`smartarts.Table`
	`smartarts.TreeNode`
	`smartarts.BulletPoints`
	`smartarts.SourceCode`
		Top-left corner (`+x` right, rows/branches grow **downward** toward `y = 0`)	`x: [x0, x0 + width]`
			`y: [y_top - height, y_top]`

Dual Solver (`get_layout()`)	`graph.*` (`ArchitectureGraph` , `LayerGraph` , `TreeGraph` , `RadialGraph` , `GridGraph`)	Exposes both center and corner coordinates on computed layout objects	`.xy` / `.local_xy` (Center) `.bottom_left` / `.local_bottom_left`

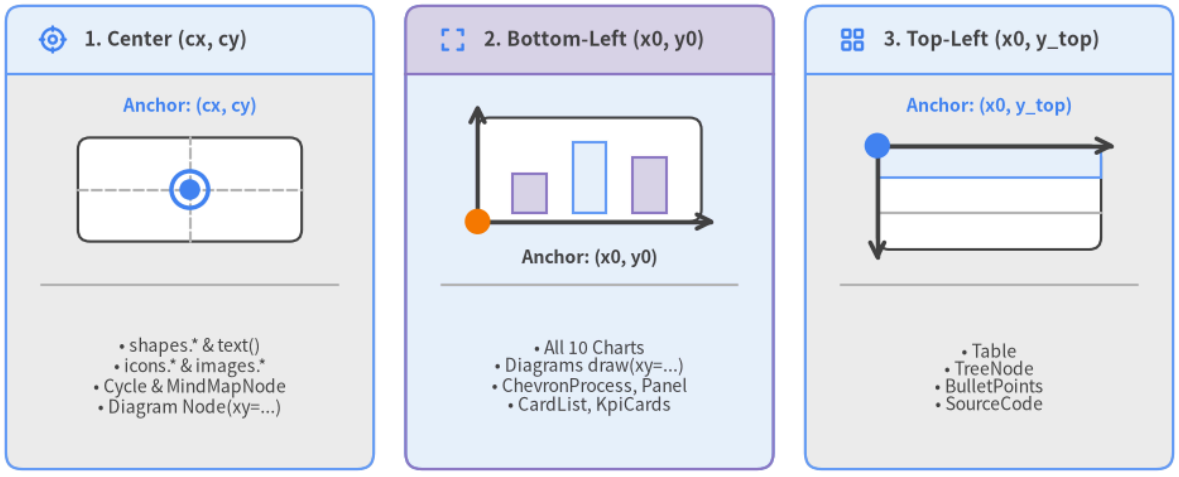


Figure 15.2: Visual Coordinate Anchor Cheat Sheet — Center (cx, cy), Bottom-Left (x0, y0), and Top-Left (x0, y_top)

15.4 Reusable Project Macros (`utils.py` & `styles.py`)

When a project grows past five diagrams, copy-pasting custom card dimensions, subtitle offsets, or CJK font declarations across Markdown blocks invites visual drift. Keep diagrams **DRY (Don't Repeat Yourself)** by separating concerns across two shared files:

- `styles.py` & `<lang>/styles.py` **(Theme & Locale Fonts)**: Configure `Colors = GoogleColors()` and `Styles = GoogleStyles()` in root `styles.py` , and place locale-specific font overrides (such as `Styles = GoogleStyles().patch_font(regular=FontJapanese.SANSSERIF_REGULAR,`

`bold=FontJapanese.SANSSERIF_BOLD)`) inside `ja/styles.py`. Drawlib hashes `styles.py` into `.drawlib/cache.db` automatically so any palette update invalidates and rebuilds dependent figures.

- `utils.py` **(Team-Wide Component Macros)**: Encapsulate recurring domain shapes—such as a standardized `service_card()` or labeled `connect()` helper—inside `utils.py` and import them via `from drawlib.utils import connect, service_card` in every chapter:

```
# Inside any chapter block or script — reusing macros from utils.py & styles.py
from drawlib.canvas import save, setup
from drawlib.styles import Styles
from drawlib.utils import connect, service_card

setup(width=128, height=42)

# Neutral supporting nodes + 1 focal PrimaryFlat hero using shared macros
service_card((26, 21), "Ingress Proxy", "Envoy :443", width=32, height=18,
style=Styles.PrimaryFlat)
service_card((96, 21), "Billing API", "Python / gRPC", width=32, height=18,
style=Styles.SecondaryNeutral)
connect((42, 21), (80, 21), label="mTLS gRPC", style=Styles.DarkBold)

save()
```

15.5 Production Pre-Flight Checklist

Before merging a documentation pull request or publishing release deliverables, run through this five-point production pre-flight checklist:

- [ ] **1. Relative Link & Asset Path Hygiene**: Every cross-page Markdown link (`02_architecture.md` , `../01_intro/index.md`) and asset path (`../_assets/logo.png` in Markdown prose; `"_assets/logo.png"` inside Python `drawlib` blocks) uses portable relative paths. Confirm zero local URI schemes or machine-specific roots such as `/abs/local/path` or `C:\local\path` , which `drawlib build` rejects at compile time.
- [ ] **2. Canvas Margins (`>= 4–6` Units) & Grid Verification (`-g`)**: Every modified diagram has been rendered with `drawlib show ... -g --no-cache` and inspected visually to verify `>= 4–6` coordinate units of outer margin on all four edges, zero text-border collisions, and `text_size >= 10.0` .
- [ ] **3.** `50%+` **Neutral Palette & Top Hero Placement**: At least half of all cards/nodes rest on calm neutral styles (`Neutral` , `PrimaryNeutral` , `SecondaryNeutral` , `White`), `WhiteBold`

is paired with `*Flat` focal nodes, and each chapter/page places a `code=fold` Top Hero diagram at **Lines 5–15** with `2+` diagrams overall.

- [ ] **4.** `i18n` **Multi-Locale Skeleton Parity**: In bilingual/multilingual projects (`langs = ["en", "ja"]`), every locale contains the identical relative file set, matching `#` / `##` / `###` heading hierarchy, matching fenced code block sequence, and `<stem>_<lang>.png` filenames for localized diagrams.
- [ ] **5. Automated Link Gate & Multi-Target Export Check**: `./<target>_src/build.sh` and `uv run drawlib serve <target>_html/ --check` exit with code `0`, `1280×920` browser views confirm above-the-fold typography harmony, compiled `PDF` / `.docx` / `.pptx` deliverables have clean page breaks, and temporary `.drawlib/scratch/` files are removed.